UNIVERSITY OF PADOVA

Embedded Real–Time Control

Laboratory report

Giulia Cassanego (2157191), Matteo Mazzocco (2163505), Davide Pillon (2156372)

Academic Year 2024-2025

Contents

1 Laboratory 1, I^2C: Line sensor and keypad

This laboratory focused on practical implementations of serial-bus communication using the I^2C protocol, interfacing external devices through the STM32's HAL (Hardware Abstraction Layer). The aim was to interface a Pololu QTR reflectance line sensor and a 16-key matrix keypad using two SX1509 I/O expanders connected to the STM32F767 microcontroller through a shared I^2C bus. The activity covered embedded systems concepts, including interrupt handling, keypad scanning, sensor polling, and dynamic control of an LED.

1.1 Theoretical background

1.1.1 I^2C & HAL Interface

The Inter-Integrated Circuit (I^2C) is a synchronous, multi-master, multi-slave serial communication protocol widely used in embedded systems for short-distance communication. It is especially useful when multiple devices share the same bus, as in the case of this laboratory. I^2C uses two bidirectional lines:

- **SCL** (Serial Clock Line): carries the clock signal generated by the master
- **SDA** (Serial Data Line): transmits and receives data between master and slaves.

Each I^2C communication involves the master initiating a transmission, specifying an address to select a slave device, followed by a read or write operation to specific registers on that device. To simplify I^2C communication, the STM32 HAL (Hardware Abstraction Layer) provides high-level functions that handle the complex low-level protocol mechanics (e.g. generating START/STOP condition, clock stretching and byte-level ACK/NACKs). The two most important functions used in this lab are:

- `HAL_StatusTypeDef HAL_I2C_Mem_Read()` - to read data from a slave register
- `HAL_StatusTypeDef HAL_I2C_Mem_Write()` - to write data to a slave register

This mechanism was used throughout the lab for example to read the current keypad status and to acquire sensor data from the QTR sensor.

1.1.2 External Interrupts on STM32

An interrupt is a hardware-triggered mechanism that suspends the main execution flow, allowing immediate response to input (e.g., key-press). In embedded systems, external interrupts are critical for handling asynchronous inputs without polling, improving efficiency and responsiveness.

STM32 microcontrollers use the EXTI peripheral (External Interrupt) to detect changes in the GPIO pins and generate interrupts accordingly. To set up external interrupts, the target GPIO pin is configured as an input with interrupt capability, and the desired trigger edge (rising, falling, or both) is selected. The pin is then mapped to the appropriate EXTI line, the interrupt is unmasked, and enabled in the NVIC with a set priority. Once configured, a signal edge on the pin triggers an interrupt that temporarily suspends normal code execution. STM32'S HAL processes this interrupt through a centralized interrupt service routine (ISR), which subsequently invokes a user-defined callback function:

- `HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)`, responsible for handling the interrupt and enabling the application layer to execute appropriate response routines.

In this laboratory, the external interrupt was configured on pin PF4, triggered by the second SX1509 I/O expander. The STM32 responded by running the callback function, which read the keypad state via I^2C to identify the pressed key.

1.2 System Configuration

- **Microcontroller:** STM32F767
- **I²C Bus:**: shared on I2C1
  - **SX1509_1** (address `0x3E`): interfaces line sensor interface.
  - **SX1509_2** (address `0x3F`): serves as the keypad controller.
- **Interrupts:**: the NINT output of `SX1509_2` is connected to PF4 on the microcontroller and configured as `GPIO_EXTI4`. This interrupt is triggered on a falling edge when a keypad button is pressed.
- **LED:** connected to PE5, used as output indicator.

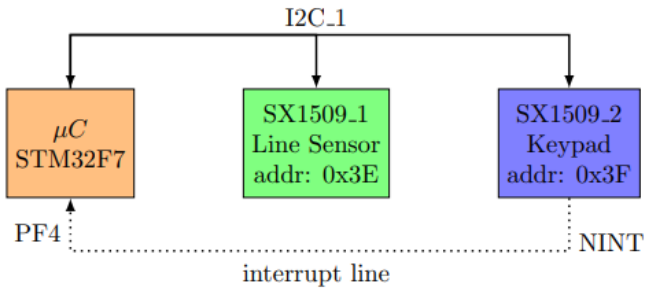


Figure 1: I²C connection scheme

1.2.1 Procedure

In the laboratory setup, the interrupt on pin PF4 was enabled by configuring the `*.ioc` project file, assigning PF4 to `GPIO_EXTI4`. The interrupt trigger was set to *External Interrupt Mode with Falling edge trigger detection*. Both internal pull-up and pull-down resistors were disabled by selecting the *No pull-up and no pull-down* option. The EXTI line interrupt was then activated within the Nested Vectored Interrupt Controller (NVIC), with the interrupt priority configured appropriately to manage interrupt preemption and nesting. Furthermore, to facilitate real-time debugging and output monitoring, the Serial Wire Viewer(SWV) protocol was enabled, allowing the use of the `printf` function for diagnostic message transmission over the debug interface. For Exercise 4 specifically, the LED connected pin PE5 was configured as `GPIO_output` within the `*.ioc` file.

1.2.2 SX1509 Register Map

The SX1509 data registers employed in this lab include:

- `0x10`: `REG_DATA_B` contains the data of the line sensor,
- `0x27`: `REG_KEY_DATA_1` contains the status of the column of the keypad,
- `0x28`: `REG_KEY_DATA_2` contains the status of the row of the keypad.

1.2.3 Keypad

A 4x4 matrix keypad is interfaced with the microcontroller through the `SX1509_2` I/O expander, which communicates via the I²C. The keypad is structured as a matrix composed of 4 rows and 4 columns, resulting in 16 unique keys. Each key press electrically connects a specific row to a specific column, enabling the identification of the key based on its position in the matrix.

To detect key presses, the SX1509 uses an internal scanning engine. The columns are configured as open drain outputs, while the rows are configured as inputs with internal pull-up resistors. When a key is pressed, it creates a path to ground between the active column and the corresponding row, causing the associated row input to be pulled low (logic '0'). In the idle state (no key pressed), all bits are high (logic '1'), resulting in register values of 0xFF (255 in decimal). When a key is pressed, the corresponding bits in both registers are pulled low (logic '0').

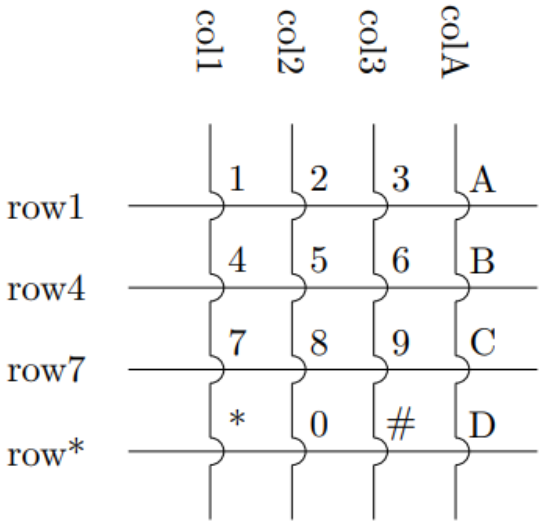


Figure 2: Keypad scheme

Each bit in the register corresponds to a specific row or column, with bit 0 (Least Significant Bit, LSB) representing the first line, and higher bits representing subsequent lines in order. In the keypad layout (see Figure 2), the LSB for the column register corresponds to column 1, followed by columns 2,3 and A, from left to right. For the rows, the LSB corresponds to row*, followed by rows 7,4 and 1, from bottom to top.

This binary information allows the software to determine the pressed key by evaluating the position of the zero bits. The microcontroller responds to this event by handling an interrupt (generated on the NINT pin connected to PF4), and reading these two registers using the HAL I^2C function `HAL_I2C_Mem_Read(&hi2c1, SX1509_I2C_ADDRX « 1, REG_KEY_DATA_X, 1, &row/&col, 1, I2C_TIMEOUT)`. The row and column indices are then mapped to a specific character using a predefined lookup table that corresponds to the keypad layout. This method provides an efficient and reliable way to detect and decode user input from a matrix keypad in embedded systems applications.

1.2.4 Line Sensor (Pololu QTR Reflectance)

The Pololu QTR reflectance sensor array is a digital infrared sensing system design to detect reflectivity, utilized in Laboratory 4 for line-following applications. It comprises multiple emitter-phototransistor pairs that measure the intensity of reflected infrared light to distinguish between reflective (e.g. white) and non-reflective (e.g. black) surfaces. In this laboratory setup, the QTR sensor interfaces with the STM32 microcontroller through an SX1509 I/O expander(I2C addres: 0X3E), enabling communication over a shared I^2C bus. Each sensor in the array operates by charging a capacitor through its phototransistor and then measuring the capacitor's discharge time. On reflective surfaces, the phototransistor saturates, causing rapid discharge and resulting in a short high-pulse duration, interpreted as a logical '1'. Conversely, dark surfaces reflect less infrared light, leading to a slower discharge and a longer high pulse, which is interpreted as a logical '0'. This binary abstraction enables digital readout of surface reflectivity:

- High (logic 1): White or reflective surface.
- Low (logic 0): Black surface.

The digital outputs of the sensor array are connected to the SX1509's input pins and read the digital state of each sensor line through its register `REG_DATA_B` (address 0x10). The STM32 polls this register using the following HAL function:
`HAL_I2C_Mem_Read(&hi2c1, SX1509_I2C_ADDR1 «1, REG_DATA_B, 1, &sensor_line, 1, I2C_TIMEOUT)`.
Each bit in the `sensor_line` variable corresponds to the state of an individual sensor element, encoding the presence and relative position of a detected line. This discrete, low-latency data stream provides reliable surface detection, forming the basis for the real-time line-following algorithm implemented in Laboratory 4.

1.2.5 LED

The LED used in this experience is connected to pin PE5 (Port E, Pin 5) on the STM32 microcontroller. The LED is directly interfaced with the GPIO through a current limiting resistor R1 (see Figure 3), which protects

the LED by controlling the current flow. The LED is controlled using STM32'S HAL functions. The most relevant function for setting the LED state is:

- `HAL_GPIO_WritePin(GPIOE, GPIO_PIN_5, GPIO_PIN_SET)`
- `HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_5)`
- `HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_5)`

This setup allows the LED to be used as a visual indicator in response to GPIO interrupts.

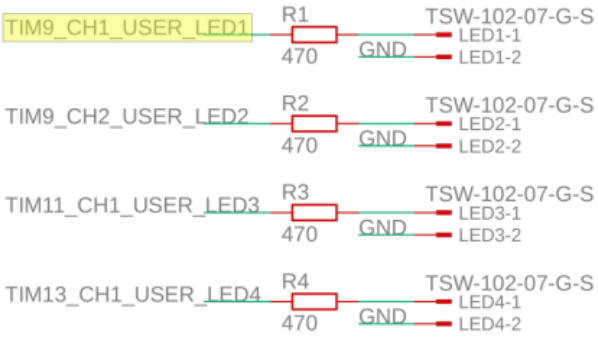


Figure 3: LED configuration

1.3 Code implementation

1.3.1 Exercise 1 - Keypad interrupt and register read

The first task involved configuring and handling external interrupts generated by the SX1509 keypad engine. Upon a key press, the SX1509 asserts the NINT line, which triggers an external interrupt on the microcontroller.

The interrupt service routine (ISR), implemented within the `HAL_GPIO_EXTI_Callback()` function (see Section 1.1.2), performs the following sequential operations:

- identification of the EXTI pin responsible for triggering the interrupt.
- reading the current keypad status by accessing the SX1509 resisters `REG_KEY_DATA_1` and `REG_KEY_DATA_2` through the `HAL_I2C_Mem_Read()` function.
- generating real-time diagnostic output using `printf()` to assist in debugging and validation.

By reading the keypad state registers directly within the ISR, latency between the physical key press and software recognition is minimized, improving system responsiveness. This method guarantees capturing the current key state immediately upon interrupt detection.

Listing 1: Read and display of the keypad register

```
1  void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
2  {
3      printf("Interrupt on pin (%d).\n", GPIO_Pin);
4
5      uint8_t col;
6      uint8_t row;
7      HAL_StatusTypeDef status_col = HAL_I2C_Mem_Read(&hi2c1, SX1509_I2C_ADDR2 <<
           1, REG_KEY_DATA_1, 1, &col, 1, I2C_TIMEOUT); //col
8      HAL_StatusTypeDef status_row = HAL_I2C_Mem_Read(&hi2c1, SX1509_I2C_ADDR2 <<
           1, REG_KEY_DATA_2, 1, &row, 1, I2C_TIMEOUT); //row
9
10     printf("col = %d\n", col);
11     printf("row = %d\n", row);
12
13     ...
14
15 }
```

1.3.2 Exercise 2 - Key decoding and display

In the second step, we interpreted the row and column data obtained from the keypad register to determine which key was pressed. According to the SX1509 specification, active key lines are indicated by cleared bits (logic level 0) within these registers.

To simplify the decoding process, we opted for a direct mapping strategy using `switch-case statements`. The reason behind this choice was to avoid the overhead and complexity of conditional logic. By directly associating each unique bit pattern with its corresponding matrix indices, we ensured fast and predictable translations. These matrix indices were then used to access the appropriate character from a predefined keypad layout, which reflects the physical arrangement of keys. This method allows for efficient and straightforward retrieval of input characters based on raw bit patterns.

Listing 2: Key decoding and display

```
const char keypadLayout[4][4] = {
    {'*', '0', '#', 'D'},
    {'7', '8', '9', 'C'},
    {'4', '5', '6', 'B'},
    {'1', '2', '3', 'A'}};

char button;

...

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
   ...
   // EXERCISE 2

   int row_index = -1, col_index = -1;

   // Each low bit (0) represents the pressed key
   // 0b11111110 = 254 -bit 0 is 0, so index 0
   // 0b11111101 = 253 -> bit 1 is 0, so index 1
   // 0b11111011 = 251 -> bit 2 is 0, so index 2
   // 0b11110111 = 247 -> bit 3 is 0, so index 3

   switch (row) {
     case 254: row_index = 0; break;
     case 253: row_index = 1; break;
     case 251: row_index = 2; break;
     case 247: row_index = 3; break;
   }

   switch (col) {
     case 254: col_index = 0; break;
     case 253: col_index = 1; break;
     case 251: col_index = 2; break;
     case 247: col_index = 3; break;
   }

   if (row_index != -1 && col_index != -1) {
     button = keypadLayout[row_index][col_index];
     printf("button = %c \n", button);
   } else {
     printf("Invalid key press: col = %d, row = %d\n", col, row);
   }

   ...

}
```

1.3.3 Exercise 3 - Line Sensor polling

In this exercise, our goal was to read the state of a line-following sensor connected through the SX1509 by accessing its `REG_DATA_B` register via the HAL function `HAL_I2C_Mem_Read()`. To achieve this, we implemented a polling loop that reads the register every 100 ms, using `HAL_Delay` to space out the reads consistently.

We chose a polling approach instead of an interrupt-driven one because the sensor can generate continuous or rapid state changes that might overwhelm the EXTI interrupt system. Polling at fixed intervals provides a controlled and reliable method to track the sensor's state without missing updates or causing excessive interrupt handling.

Listing 3: Read and display of the status of the sensor line

```
while (1)
{
    // EXERCISE 3

    uint8_t sensor_line;
    HAL_StatusTypeDef status_sens_line = HAL_I2C_Mem_Read(&hi2c1,
    SX1509_I2C_ADDR1 << 1, REG_DATA_B, 1, &sensor_line, 1, I2C_TIMEOUT);
    printf("status sensor line %d\n", sensor_line);
    HAL_Delay(100);

}
```

1.3.4 Exercise 4 - LED Blinking with dynamic frequency

This exercise explored two different methodologies for controlling the blinking frequency of an LED based on keypad input.

Easy way

A simple mapping between single-digit numeric keypad inputs ('1'-'9') and LED blink frequency (in Hz) was implemented. Key presses outside this range disable the LED by turning it off. Specifically, the key press is detected via the keypad interrupt handler `HAL_GPIO_EXTI_Callback()` (as implemented in Code 2), while the LED blinking itself is managed within the main `while(1)` loop.

Listing 4: LED blinking (easy way)

```

uint32_t period;

...

while (1)
{
   // EXERCIZE 4.1
   if(button < '1' || button > '9'){
     button = '0';
     HAL_GPIO_WritePin(GPIOE, GPIO_PIN_5, GPIO_PIN_RESET);
     continue;
   }
   uint8_t freq = (uint8_t)button - '0';
   HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_5);
   period = 1000/freq;
   HAL_Delay(period/2);

}
```

Here, the frequency is derived by converting the ASCII numeric character to its corresponding integer value. The blink period is then calculated as the inverse of the frequency, specifically as period = 1000 ms / frequency, where 1000 ms represents one second. This period is divided by two to create equal ON and OFF durations, maintaining a 50% duty cycle for the LED.

This method is straightforward and easy to implement, but it is limited to single-digit frequency ranging from 1 to 9 HZ and cannot handle multi-digit inputs, restricting the system's flexibility and precision.

Hard way

In this second approach, user interaction was improved by implementing a buffered input system that stores multiple consecutive key presses before applying a frequency update. This system stores numeric digits (0-9) in a fixed-length buffer, allowing multi-digit frequency values (up to three digits). Frequency updates are triggered only when the user presses a dedicated confirmation input # key, which signals the end of the user input. Any invalid key press resets both the buffer and the target frequency variable, preventing erroneous frequency values from being applied.

The keypad interrupt callback (`HAL_GPIO_EXTI_Callback()`) is responsible for interpreting key-press events as follows:

- Digit keys ('0'-'9'): Each valid numeric key press is appended to the input buffer, provided that the buffer length has not exceeded the maximum (3 digits).
- Confirmation key ('#'): This triggers the conversion of the buffered character sequence into an integer frequency value via `atoi()`. A flag `(freq_ready)` is set to indicate that the system should update the LED blinking frequency accordingly.
- Invalid keys: Any other key press clears the buffer and resets the frequency to zero.

Listing 5: LED blinking (hard way)

```
1
2  char input_buffer[4] = {0};  // Max 3 digits + terminator
3  uint8_t input_index = 0;
4  uint8_t freq_ready = 0;
5  uint32_t target_freq = 0;
6
7  uint32_t period;
8
9  ...
10
11 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
12 {
13     ...
14     // EXERCISE 4.2
15
16     if (row_index != -1 && col_index != -1) {
17       button = keypadLayout[row_index][col_index];
18       printf("button = %c \n", button);
19       if (button >= '0' && button <= '9') {
20         if (input_index < 3) {
21           input_buffer[input_index++] = button; // Store digit up to 3 chars
22         }
23       } else if (button == '#') {
24         input_buffer[input_index] = '\0'; // null-terminate string
25         target_freq = atoi((char*)input_buffer); // convert buffered input to
               integer
26         freq_ready = 1; // signal frequency update
27         input_index = 0; // reset buffer index
28       } else if (button < '0' || button > '9') {
29         // Reset buffer and frequency on invalid input
30         input_index = 0;
31         target_freq = 0;
32         memset((char*)input_buffer, 0, sizeof(input_buffer)); // Set all bytes in
               input_buffer to 0
33       }
34     } else {
```

```
35         printf("Invalid key press: col = %d, row = %d\n", col, row);
36     }
37
38 }
```

In the main loop, the LED blink rate is dynamically adjusted only when a valid frequency value has been flagged:

```
1  while (1)
2  {
3      /* USER CODE END WHILE */
4
5       /* USER CODE BEGIN 3 */
6      // EXERCISE 4.2
7
8      if (freq_ready && target_freq > 0) {
9          uint32_t period = 1000 / target_freq; // calculate blink period in
               milliseconds
10         HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_5);
11         HAL_Delay(period / 2); // 50% duty cycle
12     } else {
13         HAL_GPIO_WritePin(GPIOE, GPIO_PIN_5, GPIO_PIN_RESET); // LED off when idle
14         HAL_Delay(100);
15     }
16
17 }
```

This design enables the user to input a multi-digit frequency (e.g. 123 Hz), mimicking standard keypad behavior where confirmation via the # key signals the end of input. Buffering inputs before conversion improves flexibility and usability, allowing precise frequency control beyond single-digit values. Invalid inputs immediately clear the buffer to avoid unintended frequencies, enhancing robustness.

1.4 Results

The successful integration of the STM32 microcontroller with the SX1509 I/O expander demonstrated robust implementation of both interrupt-driven and polling-based communication paradigms over the I^2C bus.

Keypad Interface and Interrupt Handling:

The interrupt-driven mechanism reliably detected keypress events through the assertion of the external interrupt line (NINT) from the SX1509. Within the interrupt service routine (ISR), direct register reads of `REG_KEY_DATA_1` and `REG_KEY_DATA_1` enabled immediate acquisition of keypad state information. Bitwise decoding of these registers accurately identified the pressed keys with minimal latency. Real-time diagnostic output via serial communication validated the integrity and promptness of the interrupt handling, as illustrated in Figure 4.

Figure 4: Example of printf of the key-press

Line Sensor State Acquisition via Polling:

The line sensor's state was periodically polled every 100 ms by reading the SX1509's `REG_DATA_B` resister. This approach provided reliable and consistent sensor readings. However, while suitable for demonstration, this fixed-interval polling can cause latency and may fail to capture state changes in high-speed line-following scenarios.

LED Blinking Frequency Control:

Two LED blinking control approaches were implemented and evaluated. The first approach allowed single-digit frequency control mapped directly from keypad numeric inputs, demonstrating straightforward frequency modulation with minimal computational overhead. The second, more advanced method employed buffered multi-digit numeric input with explicit confirmation via a designated key. This buffering mechanism allowed for highly precise frequency specification, limited mainly by hardware capabilities and timing resolution. This approach significantly improved the robustness of user interaction and the accuracy of input for dynamic control application.

1.5 Conclusion

This laboratory exercise provided valuable experience in designing and implementing embedded systems that interact with external I^2C peripherals. Through the integration of the STM microcontroller with the SX1509 I/O expander, we explored key concepts such as interrupt-driven input, polling, real-time processing, and user interaction through a keypad interface.

Key insights derived include:

- The efficacy of interrupt-driven input mechanisms for latency-sensitive events, exemplified by keypad handling.
- The trade-off between polling and interrupts-driven methods when managing sensor input, emphasizing the importance of selecting appropriate strategies based on expected signal dynamics and system responsiveness requirement.
- The critical role of input buffering and validation in human-machine interfaces to ensure accurate, reliable, and user-friendly system operation.

Overall, the exercises highlighted key principles for designing an embedded system that prioritize efficient peripheral communication, real-time responsiveness, and adaptable input handling.

2 Laboratory 2, Open loop control: Camera stabilizer

In this laboratory activity, we implemented an open-loop camera stabilization system on a TurtleBot utilizing orientation data from an Inertial Measurement Unit (IMU). The main objective was to maintain a stable visual perspective by compensating for the robot's pitch and yaw motions through a pan-tilt camera mechanism.
Stabilization was achieved by estimating the robot's orientation using the BNO55 IMU sensor and applying a proportional control algorithm to compute the desired positions of two servo motors. The motors were actuated via Pulse Width Modulation (PWM) signals generated without feedback, thereby establishing an open-loop control architecture.
Data acquisition and logging was carried out using an STM32 microcontroller, with post-processing and analysis conducted in MATLAB.
This laboratory introduced key concepts such as sensor-based orientation estimation, PWM-based actuator control, proportional control strategies, and embedded data logging for offline analysis.

2.1 Theoretical background

2.1.1 Orientation estimation using IMU

The camera must maintain level orientation despite the TurtleBot's angular displacements along two axes:

- **Pitch (Tilt)** - rotation around the lateral (x) axis.
- **Yaw (Pan)** - rotation around the vertical (z) axis.

These angles are estimated using data from the BNO055 Inertial Measurement Unit (IMU), which provides 3-axis accelerometer and gyroscope measurement accessible via I^2C. STM32 HAL drivers and conversion functions process raw sensor data into physical units.
Two complementary methods are used to estimate both pitch and yaw angles: **accelerometer-based estimation** and **gyroscope-based estimation**.

Accelerometer-Based Angle Estimation

Accelerometers can estimate orientation by measuring the direction of gravity. This method works reliably for the tilt angle.

Tilt Estimation:

$$\theta = \sin^{-1}\left(\frac{a_y}{g}\right) \tag{1}$$

where:

- θ: the tilt angle (radians),
- a_y: acceleration measured along the y-axis (m/s^2),
- $g = 9.81\ m/s^2$: gravitational acceleration.

This method is effective for estimating orientation in static or slow-moving conditions. However, it becomes unreliable in dynamic environments where external forces or vibrations introduce noise, leading to incorrect estimates.

Importantly, the accelerometer is **not suitable** for pan estimation because the yaw rotation does not produce a measurable change in the gravity vector's projection on the horizontal plane. Therefore, accelerometer data cannot provide pan angle information.

Gyroscope-Based Angle Estimation

The gyroscope provides angular velocity (ω) measurements, which can be integrated over time to estimate angle displacement. Unlike the accelerometer, the gyroscope can measure both pitch and yaw angles, including rapid, dynamic movements.

Tilt Estimation:

$$\theta_g(t) = \int \omega_x(t)\, dt$$

where:

- θ_g: estimated tilt angle (radians),
- ω_x: angular velocity around the x-axis (pitch) [rad/s].

Pan Estimation:

$$\psi_g(t) = \int \omega_z(t)\, dt$$

where:

- ψ_g: estimated pan angle (radians),
- ω_z: angular velocity around the z-axis (yaw) [rad/s].

In each case, the integration is performed numerically in discrete time using sampling intervals. Once integrated, the angles are converted to degrees (by multiplying it by $180/\pi$) for use in servo control.

This approach allows real-time tracking of both tilt and pan motions and is more responsive to rapid changes in orientation, well-suited for dynamic scenarios. However, it suffers from cumulative error (drift) over time due to inherent sensor bias, which can lead to inaccurate angle estimation.

2.1.2 PWM: Pulse Width Modulation

Pulse Width Modulation (PWM) is a technique used to simulate analog output using digital signals. Instead of delivering a constant voltage, PWM controls the amount of power delivered to a device by rapidly switching the signal between high and low states. The key parameter is the duty cycle, which represents the proportion of time the signal stays high within each period.

For example, if the PWM signal has frequency of 50 HZ (i.e. a period of 20 ms), a duty cycle of 50% means that the signal is high for 10 ms and low for 10 ms within each cycle. By adjusting this ratio, it is possible to control how much effective power is delivered to a device.

PWM is widely used embedded systems because it provides an efficient way to control actuators without the need for digital-to-analog converters (DACs). In practice, STM32 hardware timers are configured to generate PWM signals with precise timing and frequency, ensuring stable and responsive control.

2.2 System Configuration

This section outlines the hardware configuration and software implementation used to estimate the TurtleBot's orientation and to control a camera stabilization system through servo motors. The system is designed to stabilize the camera in real time by counteracting the robot's pitch and yaw movements.

2.2.1 IMU: Inertial Measurement Unit

An Inertial Measurement Unit (IMU) is a sensor system that measures and reports motion-related data in three-dimensional space, including linear acceleration, angular velocity, and spatial orientation. These measurements are essential for estimating the robot's pose and enabling accurate control.

For our TurtleBot, we integrated a BNO055 sensor - a compact 9-axis IMU that combines:

- a 3-axis accelerometer for measuring linear acceleration along the x, y and z axes,
- a 3-axis gyroscope for capturing angular velocity around the same axes,
- a 3-axis magnetometer for detecting heading relative to the Earth's magnetic field.

The IMU operates in the TurtleBot's reference frame, defined as follows:

- **x-axis**: pointing to the right side of the robot,
- **y-axis**: pointing toward the rear of the robot,
- **z-axis**: pointing downward, perpendicular to the ground.

These measurements allow for the estimation of orientation using the standard Euler angles:

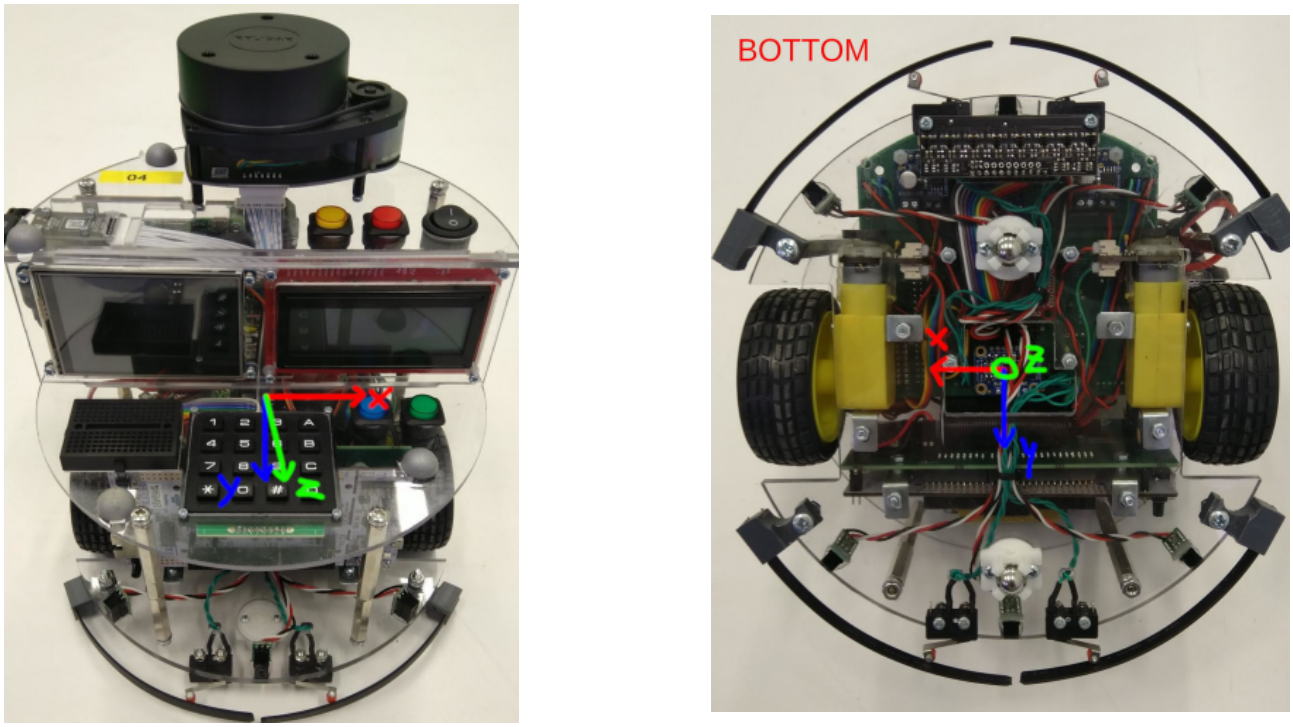
- **Pitch (Tilt):** rotation around the x-axis (camera tilts up/own)
- **Roll**: rotation around the y-axis (not used in this setup).
- **Yaw (Pan):** rotation around the z-axis (camera rotates left/right)

In the context of this experiment, only pitch and yaw, corresponding to the tilt and pan of the camera, are relevant for real-time camera stabilization. These angles are estimated in real time from IMU data and are used to drive the corresponding servo motors.

Communication with the IMU is handled via the I2C interface, using STM32 HAL drivers. Raw sensor data is processed and converted into physical units using wrapper function, such as:

- `bno55_convert_double_accel_xyz_msq()` - convert raw accelerometer data (m/s^2),
- `bno55_convert_double_accel_gyro_rps()` - convert raw gyroscope data (rad/s).

These values form the basis for the orientation estimation methods described in Section 2.1.1.



(a): Top view

(b): Bottom view

Figure 7: Turtlebot axis alignment

2.2.2 PWM-based servo motor control

A servo motor is an electromechanical actuator that allows for precise control over the rotational position of its output shaft. It comprises a small DC motor, a gear train, and an integrated feedback control system that enables accurate positioning within a predefined angular range. Unlike conventional DC motors that provide continuous rotation, servo motors are designed to rotate to a commanded angular position within specified limits, such as -45° to +45°.

In this laboratory, Pulse Width Modulation (PWM) was employed specifically to control the position of two servo motors responsible for adjusting the tilt (pitch) and pan (yaw) of the camera mounted on the TurtleBot platform. The angular position of each servo is determined by the width of the high pulse in a PWM signal with a fixed period. A wider pulse corresponds to greater angular displacement, while narrower pulses result in smaller angular movements.

The PWM signals were generated using the ST32 microcontroller's Timer 1 peripheral, which supports multiple output channels for independent control of each servo.

The configuration for this laboratory setup was as follows:

- **Tilt (Pitch) Servo** connected to **TIM1 Channel 3**,
- **Pan (Yaw) Servo** connected to **TIM1 Channel 2**.

Each channel was mapped to a dedicated GPIO pin, allowing the hardware timer to generate precise PWM signals directly to the servo control lines.

Servo actuation was managed through the STM32's HAL using the `__HAL_TIM_SET_COMPARE()` function. This function updates the timer's compare register, which in turn modulates the PWM duty cycle: `__HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_X, value)`.
Here, `value` represents the compare value that sets the hig-pulse width, and `TIM_CHANNEL_X` corresponds to the specific timer channel used for a given servo.

A software-based saturation mechanism was implemented to protect the servos from potential damage caused by PWM signals that exceed their operational limits. A custom `saturate()` function was designed to constrain PWM values within a predefined allowable range, thereby ensuring that each servo operates safely within its specified mechanical limits.

The use of two independent PWM channels allows for decoupled and simultaneous control of both camera axes. Timer-based PWM generation ensures precise signal timing, which is critical for stable servo performance.

2.3 Code implementation

2.3.1 Tilt Control

Tilt stabilization of the camera is achieved by processing real-time orientation data from the accelerometer embedded within the IMU. As detailed in Section 2.2.1, sensor measurements are acquired via the dedicated interface function:

```
1 bno055_convert_double_accel_xyz_msq(&d_accel_xyz);
```

The control strategy implements a proportional controller that computes a corrective signal to maintain the camera's tilt angle at zero relative to the horizontal plane. The proportional gain K_P was selected by empirical tuning during the experiment trials and a gain of **-1** was found to yield a stable responsive behavior.
The code for calculating the tilt angle θ_a from the accelerometer data is as follows:

Listing 6: Tilt control signal calculation

```
1  float a_y; // accelerometer's y-axis reading
2  float g = 9.81f; // gravitational constant
3  float K_p_tilt = -1; // proportional gain for tilt control
4  float theta_a = 0; // estimated tilt angle [deg]
5  float tilt = 0; // tilt control signal
6  ...
7  while (1)
8  {
9    a_y = d_accel_xyz.y; // [m/s^2]
10   theta_a = asin(a_y/g) * 180.0 / M_PI; // convert from radians to degrees [deg]
11   tilt = K_p_tilt * theta_a; // compute the control signal based on tilt angle
12   ...
13 }
```

Here, a_y is the accelerometer measurement along the y-axis and θ_a is the estimated tilt angle derived from the accelerometer data, as expressed by Equation 1.
The resulting control signal `tilt` is subsequently converted into a PWM duty cycle value to drive the servo motor responsible for tilt actuation. This mapping applies a linear scaling from the angular range $\pm 45°$ to the corresponding PWM timer compare values, centered at a neutral pulse width value of 150 timer counts:

$$\text{PWM}_{\text{value}} = \text{saturate}\left(150 + \texttt{tilt} \times \frac{50}{45},\ PWM_{\min}, PWM_{\max}\right)$$

where the scaling factor $\frac{50}{45}$ converts degrees to timer counts, and `saturate()` function constrains the PWM value within predefined bounds to prevent mechanical over-extension and potential servo damage.

This is implemented as:

Listing 7: saturate function

```
uint32_t saturate(uint32_t val, uint32_t min, uint32_t max){
    if(val<min)
        return min;
    else if(val>max)
        return max;
    else
        return val;
}
```

Servo PWM limits are defined as constants:

- `SERVO_MIN_VALUE = 100` → minimum PWM timer counts that corresponds to −45 degrees
- `SERVO_MAX_VALUE = 200` → maximum PWM timer counts that corresponds to +45 degrees

The PWM update for tilt control is then:

```
__HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3,(uint32_t)saturate((150+tilt
    *(50.0/45.0)), SERVO_MIN_VALUE, SERVO_MAX_VALUE)); // tilt
```

This approach ensures precise, bounded control of the servo motor's angular position, enabling stable tilt compensation in an open-loop configuration based on IMU orientation estimates.

2.3.2 Pan Control

Pan stabilization of the camera is achieved by estimating the orientation around the vertical (z) axis using gyroscope data from the IMU. As described in Section 2.2.1, gyroscope data is acquired through the same interface used for tilt control:

```
bno055_convert_double_gyro_xyz_rps(&d_gyro_xyz);
```

The control strategy employs a proportional controller that generates a corrective signal based on the estimated pan angle, aiming to align the camera orientation with the initial heading. The proportional gain K_P was selected through empirical tuning; a gain value of **-1** was selected as it provided a satisfactory trade-off between responsiveness and stability.
Unlike tilt, which can be estimated directly from accelerometer data, the pan angle ψ_g must be obtained by numerical integration of the gyroscope's angular velocity ω_z. Assuming an initial pan angle of zero, the integration accumulates the relative rotation over time.
The code for calculating the pan angle ψ_g from the gyroscope data is as follows:

Listing 8: pan control signal calculation

```
float w_z;  // gyroscope's z-axis
float K_p_pan = -1; // proportional gain for pan control
float phi_g = 0; // estimated pan angle [deg]
float dt = 0.02; //for integration of w_z
float pan = 0; // pan control signal

...
while (1)
{
  w_z = d_gyro_xyz.z;     // [rad/s]
  phi_g += w_z * dt * 180.0 / M_PI;  // integrate angular velocity over time to
      estimate pan angle [deg]
  pan = K_p_pan * phi_g; // compute the control signal based on pan angle
  ...
}
```

The computed control signal `pan` is linearly scaled and converted to a PWM duty cycle value to command the pan servo motor, employing the same mapping used for tilt control:

$$\text{PWM}_{\text{value}} = \text{saturate}\left(150 + \texttt{pan} \times \frac{50}{45},\ PWM_{\text{min}}, PWM_{\text{max}}\right)$$

The corresponding implementation is:

```
__HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, (uint32_t)saturate((150+pan
    *(50.0/45.0)), SERVO_MIN_VALUE, SERVO_MAX_VALUE)); // pan
```

This open-loop control approach allows for real-time adjustment of the camera's pan angle. Although the integration of gyroscope data is susceptible to long-term drift, the method provides reliable performance for short-duration or limited-range movements within defined operational constraints.

2.3.3 Data logging

The system supports data logging via two communication interfaces:

- **Serial interface** through USB (used in this laboratory)
- **Wifi datalogger** using UDP communication

MATLAB:

Serial datalogger:

The serial datalogger can be interfaced directly from MATLAB using the command:

```
data = serial_datalog ('COMx', {'3*single','1*single'}, 'baudrate', 115200);
```

where the string `'3 * single'` corresponds to three single precision floating point accelerometer / gyroscope readings, and `'1 * single'` corresponds to the single control signal value.
The `serial_datalog` function automatically detects the active COM port of the STM32 board when invoked alongside MATLAB's `seriallist` routine, assuming the STM32 is the only connected device. The acquired data are displayed in real-time on MATLAB figure and simultaneously buffered for post-processing. Upon closing the visualization figure, the buffered data are returned as a structured variable `data` with fields:

- `data.time`: timestamps corresponding to each received sample
- `data.out`: a cell array where each cell contains matrices of logged signals; rows correspond to signal components and columns to sample instances

For example, the command `y = data.out{2}(2,:)` extracts the second components of the second logged signal.

Wifi datalogger:

The WiFi datalogger can be used in MATLAB with the following command:

```
data = udp_datalog ('IP_address', port, packet_specification);
```

where `'IP_address'` is the IP of the STM32 board and `port` is the UDP port it listens on. The `packet_specification` works exactly like in the serial version, describing the structure of the transmitted data.
The `udp_datalog` function connects to the robot over WiFi using UDP communication. Data is displayed in real-time in a MATLAB figure and buffered. Once the figure is closed, the buffered data is returned as a structured variable `data`, with the same field as the serial datalogger (`data.time`, `data.out`).

STM32:

On the embedded STM32 platform, data logging is facilitated through a user-defined `struct` representing the data packet. In this laboratory, the data structure for logging the data is defined as:

Listing 9: datalog structure

```
struct datalog
{
    float accel_x, accel_y, accel_z;
    //float gyro_x, gyro_y, gyro_z;
    float th, control_signal_tilt;
    //float ph_g, control_signal_pan;
} logger_data;
```

Within the main application loop, sensor data and control signals are continuously updated and transmitted via the configured UART interface:

Listing 10: UART3 serial communication

```
/* declare an ertc_dlog struct */
struct ertc_dlog logger;

int main(void)
{
  /* USER CODE BEGIN 2 */
  logger.uart_handle = huart3; // select UART3 for serial communication
  /* USER CODE END 2 */

  while (1)
  {
    /* USER CODE BEGIN 3 */

    logger_data.accel_x = d_accel_xyz.x;
    logger_data.accel_y = d_accel_xyz.y;
    logger_data.accel_z = d_accel_xyz.z;
    //logger_data.gyro_x = d_gyro_xyz.x;
    //logger_data.gyro_y = d_gyro_xyz.y;
    //logger_data.gyro_z = d_gyro_xyz.z;*/

    logger_data.control_signal_tilt = tilt; // computed tilt control signal
    //logger_data.control_signal_pan = pan; // computed pan control signal
    logger_data.th = theta_a; // tilt angle from acclerometer
    //logger_data.ph_g = phi_g; // pan angle from gyroscope

    ertc_dlog_update(&logger);//check if a connection is established
    ertc_dlog_send(&logger, &logger_data, sizeof(logger_data));//send data to
       MATLAB

    /* USER CODE END 3 */

  }

]
```

The `ertc_dlog_update()` function checks for an active connection, while `ertc_dlog_send()` transmits the current data structure to MATLAB.

This integrated data logging framework enables systematic acquisition and real-time monitoring of sensor measurements and control signals, thereby supporting detailed diagnostics and quantitative evaluation of system performance.

2.4 Results

2.4.1 Exercise 1 : Tilt Stabilization

The tilt stabilization system, based on a proportional control law using real-time accelerometer data from the IMU, demonstrated effective compensation of manual tilt disturbances applied to the TurtleBot platform. Specifically, the accelerometer's y-axis measurements were processed to estimate the tilt angle, which was then used to generate a control signal driving the tilt servo motor.

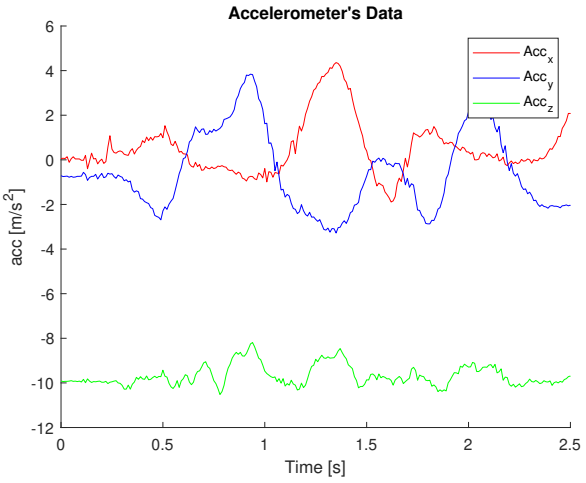


Figure 8: Accelerometer data

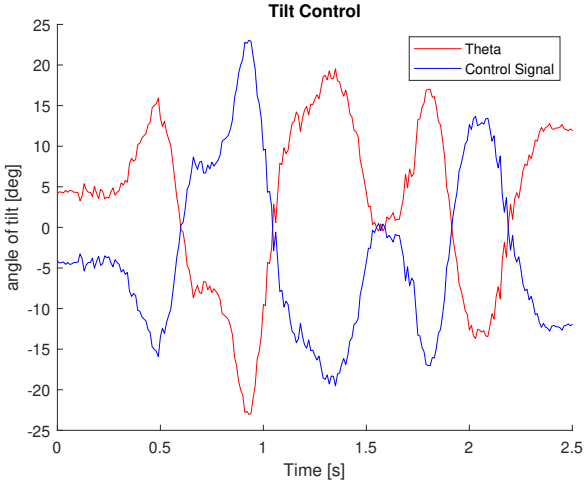


Figure 9: Tilt control signal

Figure 8 shows the raw accelerometer data along the three axes (x, y, and z) over time. The y-axis data clearly reflects changes in tilt relative to gravity, while the x and z axes exhibit dynamic variations due to movement and vibration. This confirms the IMU's capability to accurately capture relevant motion data for orientation estimation.
The corresponding tilt control signal and the estimated angle are shown in Figure 9. The control signal effectively tracks and counteracts the tilt changes, demonstrating a responsive and consistent actuation of the servo motor in reaction to forward and backward tilting of the TurtleBot. This open-loop configuration successfully maintains the camera's horizontal orientation by compensating for tilt disturbances in real time. Despite its simplicity, the system shows stable and smooth behavior, validating the effectiveness of the implemented proportional controller.

2.4.2 Bonus: Smooth Pan Control

Due to limitations in real-time data acquisition during the laboratory session, the gyroscope measurements and the corresponding pan control signals were erroneously obtained from an independent simulation rather than from simultaneous live execution. Despite this constraint, the results demonstrate the expected dynamic behavior of the open-loop pan control strategy.

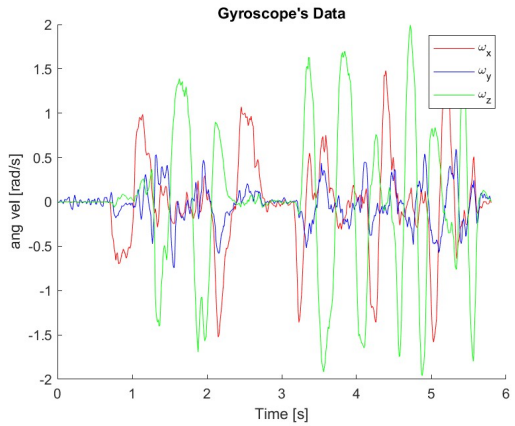


Figure 10: Gyroscope data

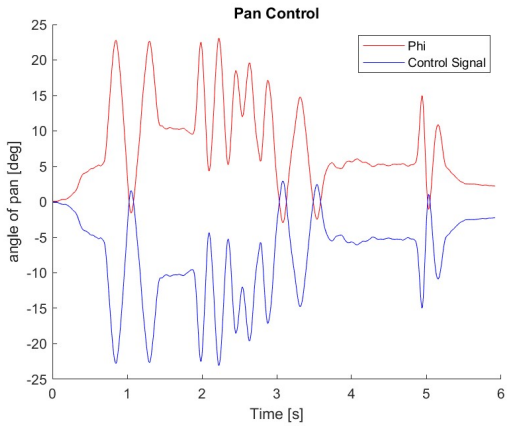


Figure 11: Pan control signal

As shown in Figure 10, the gyroscope's z-axis output provides the angular velocity required to estimate the yaw (pan) angle. This velocity is integrated over time to obtain a relative orientation estimate. The control signal, depicted in Figure 11, is generated through a proportional controller that adjusts the servo position in accordance with the estimated pan angle.
The separately control signal exhibits a smooth and consistent trend, in line with the expected response to gradual changes in orientation. The proportional gain effectively regulates the responsiveness of the servos to estimated angular displacements. Although integration-based methods are inherently prone to drift over time, open-loop control remains effective within short durations and limited angular ranges, where accumulated error does not significantly impact performance.

2.5 Conclusion

The implemented tilt and pan control strategies demonstrate foundational capabilities for camera stabilization using IMU data. The tilt controller effectively achieves open-loop stabilization through accelerometer-based angle estimation combined with the proportional controller. The pan controller approach, evaluated through separate acquisition of gyroscope data and control signals confirms that integration of angular velocity measurements with proportional control can produce smooth orientation adjustments.

However, these approaches exhibit limitations: accelerometer-only tilt estimation is susceptible to high-frequency noise and transient disturbances, while gyroscope integration for pan is prone to drift over time. To address these challenges, sensor fusion techniques, such as the complementary filter, offer a promising direction for improving orientation estimation and control performance. By combining gyroscope and accelerometer data, such methods can mitigate drift and noise, enabling more accurate tilt and pan stabilization. Given their low computational overhead, complementary filters are particularly well suited for real-time embedded applications and represent a practical enhancement for future development.

3 Laboratory 3, Motor control

The objective of Laboratory 3 was to implement a PI (Proportional-Integral) controller for the speed regulation of DC motors mounted on the TurtleBot. The control algorithm was deployed on an STM32F7 microcontroller, utilizing real-time feedback from rotary encoders. The challenge was to regulate the angular speed of the DC motors of the TurtleBot, considering non-idealities such as mechanical inertia, friction, motor nonlinearities, and encoder quantization. The experiment aimed to evaluate and compare two different driving techniques: **Forward/Brake** and **Forward/Coast**, analyzing their influence on system response. Additional improvements, such as keypad input for setting speed references, were considered as bonus tasks.

3.1 Theoretical background

3.1.1 DC Motor Model

The DC motor used in this experiment converts electrical energy into mechanical rotation. It consists of a stator providing a magnetic field and a rotor which rotates when current passes through its windings. The interaction between the magnetic fields produces torque, which is proportional to the armature current.
The electrical behavior of the motor follows:

$$V_a(t) \;=\; R_a\, i_a(t) + L_a\, \frac{di_a(t)}{dt} + K_b\, \omega(t) \tag{2}$$

The mechanical dynamics are governed by:

$$J\, \frac{d\omega(t)}{dt} \;=\; K_t\, i_a(t) - b\, \omega(t) \tag{3}$$

Using the Laplace transform on Equation (2) and (3) and assuming $K = K_t = K_b$:

$$G(s) = \frac{\Omega(s)}{V_a(s)} = \frac{K}{L_a J s^2 + (R_a J + L_a b)s + (R_a b + K^2)} \tag{4}$$

Where:

Symbol	Description [Unit]
$i_a(t)$	Armature current [A]
$\omega(t)$	Angular velocity [rad/s]
J	Moment of inertia [kg·m^2]
b	Viscous friction coefficient [Nms/rad]
R_a	Armature resistance [Ω]
L_a	Armature inductance [H]
K_b	Back EMF constant [V/(rad/s)]
K_t	Torque constant [Nm/A]

Table 1: List of motor model variables and parameters

3.1.2 PI Controller

To regulate the motor speed, a PI controller is implemented in discrete time. At each sampling instant T_s the error between the reference speed $r(t)$ and the measured speed $y(t)$ is computed. The control signal is then given by:

$$u(t) \;=\; K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau \tag{5}$$

3.1.3 Motor driver and H-birdge

In a typical embedded motor control system, the microcontroller alone cannot directly drive a DC motor, due to its limited voltage and current capabilities. To address this, an external component known as a motor driver is employed. In our setup, the DRV8871 motor driver by Texas Instruments was used.

The DRV8871 is specifically designed for controlling brushed DC motors and supports bidirectional control through an integrated H-bridge circuit. An H-bridge is an arrangement of switches (usually transistors) that allows current to flow through the motor in either direction. This enables forward and reverse rotation, as well as different braking behaviors. The scheme of the DRV8871 H-bridge is shown in Figure 12.

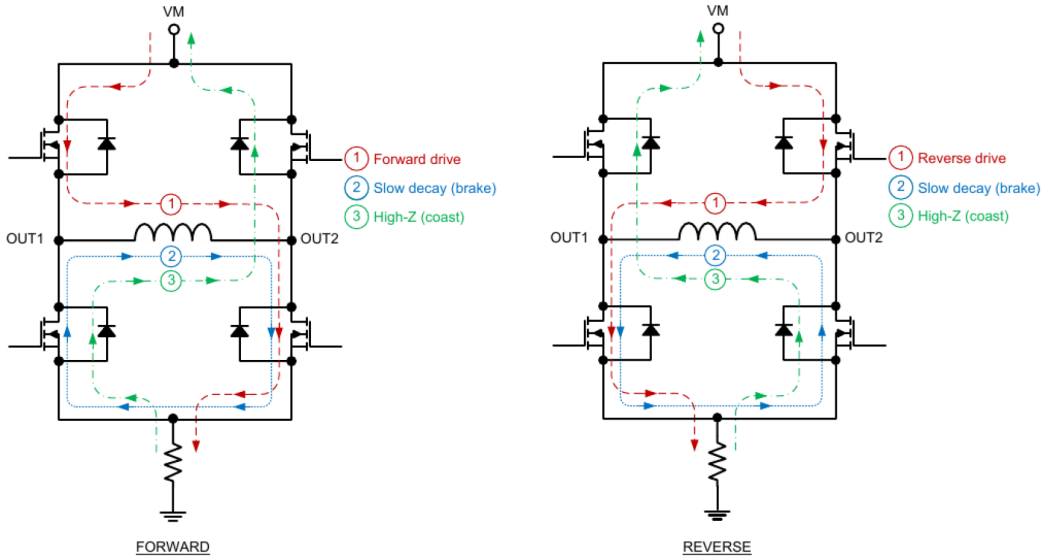


Figure 12: DRV8871 H-bridge

The control logic of the H-bridge is driven by two input pins (IN1 and IN2), which are typically connected to the PWM outputs of the microcontroller. The behavior of the motor depends on the logic levels applied to these inputs, as shown in the table below:

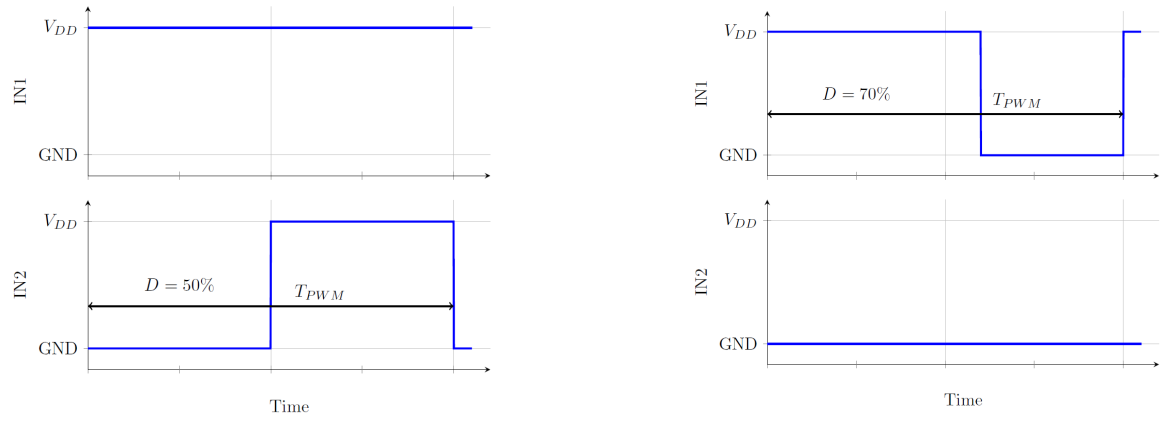
IN1	IN2	OUT1	OUT2	Mode
0	0	Hi-Z	Hi-Z	Coast
0	1	L	H	Reverse
1	0	H	L	Forward
1	1	L	L	Brake

Table 2: DRV8871 H-Bridge control logic

Three PWM control strategies can be used with the DRV8871:

- Continuous Drive (IN1 = 1, IN2 = 0 or vice versa) – Motor runs at full speed in the chosen direction.
- PWM with Forward and Brake – The motor alternates between driving and braking, which gives finer speed control and is recommended by the datasheet.
- PWM with Forward and Coast – The motor alternates between driving and coasting, which can be more energy-efficient but results in slower deceleration.

The duty cycle behaviours of the forward and brake and the forward and coast methods are shown in Figure 13.



(a) Forward and Brake duty

(b) Forward and Coast duty

Figure 13: Comparison Between Forward/Brake and Forward/Coast duty cycle behaviours

3.1.4 Position encoder

A position encoder is a sensor that converts mechanical motion into electrical signals, allowing the measurement of angular position and speed. In our setup, we used a rotary incremental encoder.

This type of encoder generates a series of pulses as the shaft rotates. Each pulse corresponds to a small increment in position. The encoder used in our system is of the quadrature type, which provides two output signals, typically called A and B. These signals are square waves shifted by 90 degrees in phase.

The phase shift between signals A and B enables the detection of the direction of rotation: depending on which signal leads, the system can determine whether the motor is turning clockwise or counterclockwise.

By counting the number of pulses, the system can estimate the motor's position. Additionally, by measuring how many pulses occur over a known time interval, it is possible to calculate the angular speed of the motor. The code implementation of this approach will be presented later in this report.

3.2 System Configuration and Controller Design

3.2.1 PWM setting

The theoretical background related to PWM can be found in Section 2.1.2. The voltages applied to the inputs should have at least 800 ns of pulse width to ensure detection. Typical devices require at least 400 ns. If the PWM frequency is 200 kHz, the usable duty cycle range is 16% to 84%. The configuration of the timer used to generate the PWM signal (TIM8) is as follows:

- Clock source: APB1 Timer clock at 96 MHz.
- Prescaler (PSC): 959, which leads to a timer frequency:

$$f_T = \frac{96 \cdot 10^6}{959 + 1} = \frac{96 \cdot 10^6}{960} = 10^5 \,\text{Hz} = 100 \,\text{kHz}$$

- Counter period (Auto-Reload Register): 399, resulting in a PWM frequency:

$$f_{\text{PWM}} = \frac{f_T}{ARR + 1} = \frac{10^5}{400} = 250 \,\text{Hz} \Rightarrow T_{\text{PWM}} = 4 \cdot 10^{-3} \,\text{s}$$

We assume a linear relationship between the duty cycle and the voltage applied to the motor. The voltage is expected to be within the range $[0, V_{\text{PowerSupply}}]$. Depending on the source:

$$V_{\text{PowerSupply}} \approx \begin{cases} 8 \,\text{V} & \text{if powered by battery } (V_{\text{BAT}}) \\ 12 \,\text{V} & \text{if powered by wall supply } (V_{\text{Wall}}) \end{cases}$$

3.2.2 Peripheral List

- TIM3 (16bit, general-purpose timer), channel 1 and channel 2: Encoder for motor 1;
- TIM4 (16bit, general-purpose timer), channel 1 and channel 2: Encoder for motor 2;
- TIM6 (16bit, basic timer): Provides the clock for the controller;
- TIM8 (16bit, advanced timer), channel 1 and channel 2: PWM for motor 1;
- TIM8 (16bit, advanced timer), channel 3 and channel 4: PWM for motor 2.

The block scheme of the hardware implementing the motor control is shown in the Figure 14.

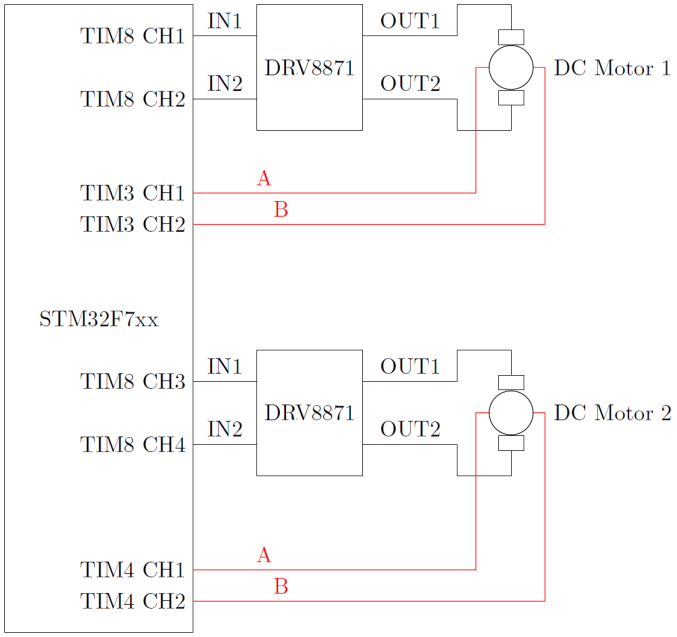


Figure 14: Connection scheme for the timers

Note: in Figure 14 we can be see the 2 channels used for acquire the A and B signals of the encoder described in Section 3.1.4.

3.2.3 Motor and Encoder

- The rover is equipped with a DC motor that comes along with an encoder;
- Electrical characteristics for the motors can be found in the datasheet;
- The module encompasses a gearbox with a 120:1 ratio, which means that for every round performed by the wheel, the motor rotates 120 times.

Encoder Configuration and Filtering

In our setup, each motor is equipped with a quadrature incremental encoder that outputs two square wave signals (A and B), shifted by 90 degrees. These signals are connected to timer input channels TIM3 for motor 1 and TIM4 for motor 2 (see Figure 14), which are configured in encoder mode. The encoder generates 8 pulses per revolution on the motor side in 1X mode. Considering the 120:1 gearbox ratio, this translates to:

- **1X mode:** $8 \times 120 = 960$ PPR
- **2X mode:** $16 \times 120 = 1920$ PPR
- **4X mode:** $32 \times 120 = 3840$ PPR

The chosen counting mode is **4X**, meaning all rising and falling edges of both A and B channels are counted. The 4X mode was selected for this implementation because it offers the finest resolution and halves the quantization error compared to 2X mode. A higher resolution allows for more precise speed estimation, which directly improves control accuracy.

The Counter period (ARR) of the encoder timers was set to match the number of pulses per revolution.

This configuration ensures that the timer overflows exactly once per wheel revolution, simplifying the count-difference calculation. Overflow and underflow conditions are explicitly handled in the control callback to maintain accurate speed readings in both directions of rotation.

3.2.4 Controller design - PI Gains Selection

The proportional (K_p) and integral (K_i) gains used in the PI controller were selected through a trial-and-error approach. Initial low values were chosen, and then gradually increased while monitoring the system's response to step changes in the reference speed.

The objective was to achieve a good trade-off between rise time, overshoot, and steady-state error. After several iterations and tests on both motors, the gain values in Table 11 were selected:

- $K_p = 0.5$;
- $K_i = 6$.

These values provided satisfactory performance in the tested scenarios, including both Forward/Brake and Forward/Coast drive modes.

3.3 Code implementation and control

In the following code snippet, all key variables used for motor control are declared. Each motor (Motor 1 and Motor 2) is assigned its own dedicated set of variables. These variables include the measured speed (`TIMx_speed`), the reference speed (`reference_speed`), the speed error (`speed_error`), and the control terms for the PI controller (`u_p`, `u_int`, `u_pi`, `Kp`, `Ki`), along with the PWM duty cycle (`duty[x]`).

In addition, a `struct datalog` is defined to store relevant variables such as the control signal (`u1`, `u2`, `u3`) and the measured speed (`w1`, `w2`, `w3`) for post-processing and analysis. This structure is used in conjunction with MATLAB data logging to collect time-series data for both motors during operation.

Listing 11: Global variables

```
struct datalog {
    float u1, u2, u3;
    float w1, w2, w3;
} data;

// motor 1
float TIM3_speed;
float reference_speed = 0; //RPM
float speed_error;
static float u_int;
float u_p;
float u_pi;
float Kp = 0.5;
float Ki = 6;
uint32_t TIM3_CurrentCount;
int32_t TIM3_DiffCount;
static uint32_t TIM3_PreviousCount = 0;
float duty;

// motor 2
float TIM4_speed;
float reference_speed2 = 0; //RPM
float speed_error2;
static float u_int2;
float u_p2;
float u_pi2;
float Kp2 = 0.5;
```

```
28 float Ki2 = 6;
29 uint32_t TIM4_CurrentCount;
30 int32_t TIM4_DiffCount;
31 static uint32_t TIM4_PreviousCount = 0;
32 float duty2;
```

The core of the control logic is implemented inside the `HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)` function, which is periodically called through the timer interrupt.

The function `HAL_TIM_PeriodElapsedCallback()` has already been introduced and described in Section 1.1.2. In this implementation, the callback manages both motors independently, computing speed measurements from encoders, evaluating the control action using a PI controller, updating the PWM duty cycles, and logging data for analysis.

The following code snippet focuses on the encoder-based speed measurement for Motor 1.

Listing 12: Encoder speed calculations

```
1  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
2  {
3      //MOTOR 1
4      // ENCODER CALCULATION
5      TIM3_CurrentCount = __HAL_TIM_GET_COUNTER(&htim3);
6      if(__HAL_TIM_IS_TIM_COUNTING_DOWN(&htim3))
7      {
8          if(TIM3_CurrentCount <= TIM3_PreviousCount)
9              TIM3_DiffCount = TIM3_CurrentCount - TIM3_PreviousCount;
10         else
11             TIM3_DiffCount = -((TIM3_ARR_VALUE+1) - TIM3_CurrentCount) -
                   TIM3_PreviousCount;
12     }
13     else
14     {
15         if(TIM3_CurrentCount >= TIM3_PreviousCount)
16             TIM3_DiffCount = TIM3_CurrentCount - TIM3_PreviousCount;
17         else
18             TIM3_DiffCount = ((TIM3_ARR_VALUE+1) - TIM3_PreviousCount) +
                   TIM3_CurrentCount;
19     }
20     TIM3_PreviousCount = TIM3_CurrentCount;
21     TIM3_speed = (TIM3_DiffCount * 60) / (TIM3_ARR_VALUE * TS); // [rpm]
22     speed_error = reference_speed - TIM3_speed;
```

The code in Listing 12 performs encoder-based speed computation for Motor 1 using Timer 3. Since the encoder is configured in quadrature mode, the timer's counter increases or decreases depending on the direction of rotation. To compute the motor speed, the difference in counts between two consecutive interrupts is measured and scaled appropriately.

However, due to the finite size of the timer register, the counter value wraps around when reaching its maximum or minimum, a phenomenon known as "overflow" (when counting up) or "underflow" (when counting down). If not handled correctly, this wrap-around can lead to large, incorrect differences and thus invalid speed measurements.

To manage this, the code explicitly checks the counting direction using:

- `__HAL_TIM_IS_TIM_COUNTING_DOWN(&htim3)`: returns true if the timer is currently counting down (i.e., the motor is rotating in reverse).

Then it compares the current counter value with the previous one:

- If the direction and values are consistent (no overflow), it computes the difference directly.
- If the counter goes in overflow, it applies corrective logic using the value `TIM3_ARR_VALUE`, which defines the timer's auto-reload register (i.e., its maximum value before rolling over).

The corrected count difference `TIM3_DiffCount` is then converted to RPM using the formula in line 21. Finally, the speed error is computed as the difference between the desired reference speed and the measured speed.

This value will be used as the input to the PI controller in the next step.

The next snippet shows the PI controller implementation.

Listing 13: PI controller

```
1     u_int += Ki * TS * speed_error;
2     u_p = Kp * speed_error;
3     u_pi = u_int + u_p;
```

The integral term accumulates the error over time, while the proportional term reacts to the current error. The final control output `u_pi` is used to adjust the PWM duty cycle sent to the motor. The tuning of the controller gains `Kp` and `Ki` is discussed in Section 3.2.4. Note that the integral term `u_int` is not bounded or clamped, which may lead to integrator windup if the error persists for a long time. This can be mitigated by adding saturation or anti-windup logic, though it is not implemented in this version. This part of the code applies a discrete PI control law to regulate the motor speed.

The following section handles data logging (as detailed in Section 2.3.3):

Listing 14: Data logging and status LED

```
1     static int kLed = 0;
2     /* Speed ctrl routine */
3     if(htim->Instance == TIM6)
4     {
5         // SEND VALUES TO STRUCTURE FOR MATLAB
6         data.u1 = TIM3_speed;          // speed [rpm]
7         data.u2 = speed_error;         // speed error [rpm]
8         data.u3 = u_pi;                // control action [V]
9
10        //ertc_dlog_send(&logger, &data, sizeof(data));
11        // Indicate that the program is running
12        if(++kLed >= 10)
13        {
14            kLed = 0;
15            HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
16        }
17    }
```

When `htim->Instance == TIM6`, the code logs the most relevant variables for Motor 1: the measured speed, the speed error, and the control output. These values are stored in a structure called `data` (see Listing 11), which is used for communication with MATLAB for visualization and analysis. The LED toggle acts as a simple heartbeat signal, blinking every 10 callback cycles to indicate the system is running correctly.

The last portion of the function computes the PWM duty cycle and updates the hardware registers accordingly.

Listing 15: PWM duty cycle and motor direction

```
1     duty = u_pi * V2DUTY;      // CONVERTS THE CONTROL ACTION INTO DUTY CYCLE (
          maximum duty is 399, corresponding to 12V or 8V with the battery)
2     //duty = 399;              // max value
3     if(duty>=TIM8_ARR_VALUE)     // Limits the duty cycle, and thus the voltage (
          to 100%)
4     {
5         duty = TIM8_ARR_VALUE;
6     }
7
8     /* calculate duty properly */
9     if ( duty >= 0)
10    {// rotate forward
11     /* alternate between forward and coast */
```

```
12          //__HAL_TIM_SET_COMPARE (&htim8, TIM_CHANNEL_1, (uint32_t)duty);
13          //__HAL_TIM_SET_COMPARE (&htim8, TIM_CHANNEL_2, 0);
14       /* alternate between forward and brake , TIM8_ARR_VALUE is a define */
15          __HAL_TIM_SET_COMPARE (&htim8 , TIM_CHANNEL_1 , (uint32_t)TIM8_ARR_VALUE
                );
16          __HAL_TIM_SET_COMPARE (&htim8 , TIM_CHANNEL_2 , (uint32_t)(
                TIM8_ARR_VALUE - duty));
17      } else { // rotate backward
18          __HAL_TIM_SET_COMPARE (&htim8, TIM_CHANNEL_1, 0);
19          __HAL_TIM_SET_COMPARE (&htim8, TIM_CHANNEL_2, (uint32_t)-duty);
20      }
21
22      //MOTOR 2
23      // ENCODER CALCULATION
24      . . .
25  }
```

Here, the control signal `u_pi` is converted into a PWM duty cycle using the constant `V2DUTY`. To prevent overvoltage, the duty cycle is limited to a maximum value defined by `TIM8_ARR_VALUE`. Depending on the sign of the control signal, the motor is commanded to rotate forward or backward.

The two drive modes are: **Forward and Coast** and **Forward and Brake**. These can be selected by commenting or uncommenting the corresponding `__HAL_TIM_SET_COMPARE` lines (lines). This function sets the compare value for a given timer channel, which directly affects the PWM output used to drive the H-bridge. The **Forward and Brake**: one terminal receives the PWM signal and the other is connected to the opposite voltage level, effectively shorting the motor terminals during the off-time. This provides active braking and allows for faster deceleration of the motor. The **Forward and Coast** method: one motor terminal receives the PWM signal and the other is set to zero.

As shown in the hardware connection in Section 3.2.2, Motor 2 is controlled by channels 3 and 4 of Timer 4. Thus, updating the compare functions accordingly (i.e., replacing 1→3 and 2→4) allows for full control of the second motor using the same PI control structure.

Therefore the same control logic of Motor 1 is duplicated further down in the same callback function to control Motor 2. The only differences are:

- Variables related to Timer 3 (`TIM3_`) are replaced by Timer 4 equivalents (`TIM4_`).
- PWM output channels are shifted from channel 1 and 2 (releted to timer 3) to channel 3 and 4 (releted to timer 4).

Exercise Bonus 3.1.4: Setting the reference speed using the keypad

As an additional feature, dynamic speed reference adjustment via keypad was implemented. The code structure utilizes the same keypad hardware and I2C communication protocol introduced in Laboratory 1, where the keypad was originally used to set the LED blinking frequency. In Laboratory 1 (see List 5), the assignment required the user to input a frequency value (e.g., `125#`) to dynamically control the LED blinking rate. Building upon that logic, in this laboratory, the keypad was repurposed to allow real-time motor speed adjustments by entering a numeric value followed by the `#` symbol.

Listing 16: Exercise Bonus 3.1.4: Setting the reference speed using the keypad

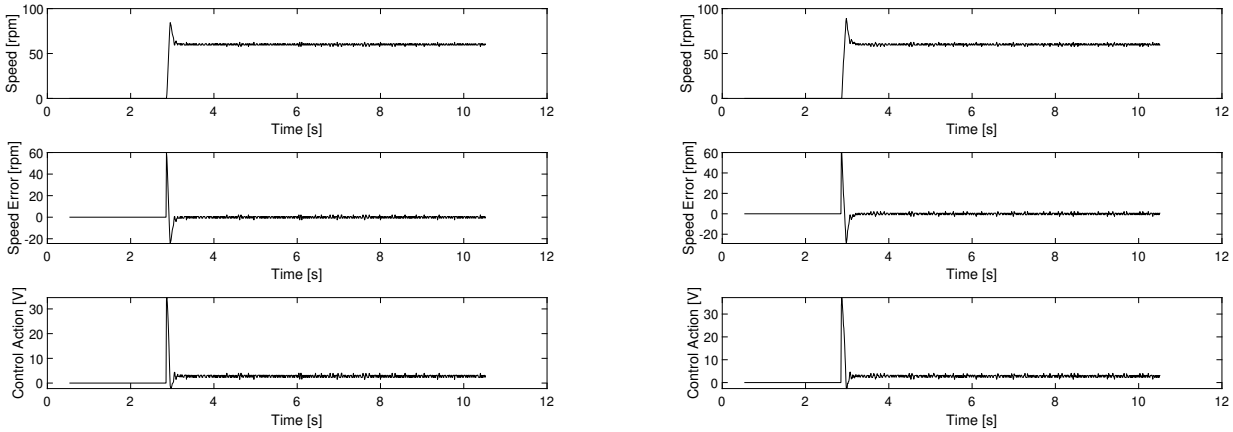
```
1   /* USER CODE BEGIN WHILE */
2   while (1)
3   {
4     /* USER CODE END WHILE */
5
6     /* USER CODE BEGIN 3 */
7       ertc_dlog_update(&logger);
8
9       // BONUS EX 1
10          if (speed_ready && target_speed <= 120 ){
11            //KEYPAD (only positive speed)
12              reference_speed2 = target_speed;
13              reference_speed = target_speed;
```

```
14              } else {
15                  reference_speed2 = 0;
16                  reference_speed = 0;
17              }
18          }
19      }
20      /* USER CODE END 3 */
```

The code snippet above implements the logic for reading the speed reference value entered via the keypad and updating the motor speed reference accordingly. The variable `target_speed` holds the value parsed from the keypad input, while the `speed_ready` flag indicates whether a complete and valid number has been received (i.e., the user has pressed the `#` key). The `if` condition ensures that only positive speed values up to 120 are accepted, reflecting the system constraint. When the input is valid, the variables `reference_speed` and `reference_speed2` are updated with the desired target value, which is then used in the motor control loop. If the input is invalid or exceeds the allowed range, both reference variables are set to zero, effectively stopping the motor to prevent unintended behavior.

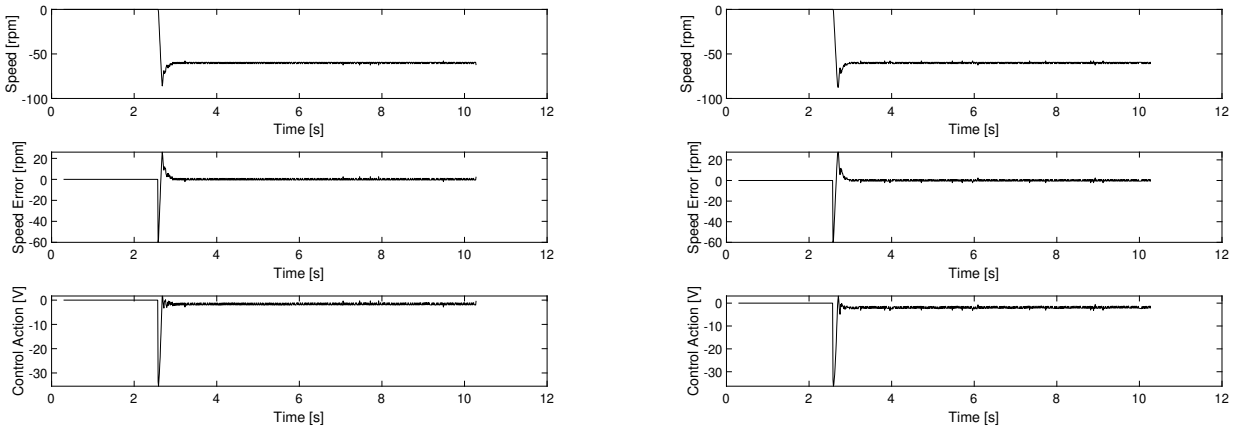
3.4 Results and observations/conclusions

The performance of the PI controller was evaluated using both Forward/Brake and Forward/Coast actuation strategies, for two step reference speeds: $+60$ and -60 rpm. The experiments were conducted on both motors and the following signals were logged via MATLAB: Measured angular speed $[rpm]$, Speed error $[rpm]$ and Control effort $[V]$. Each test lasted approximately 10 seconds to ensure the transient and steady-state behavior were visible.



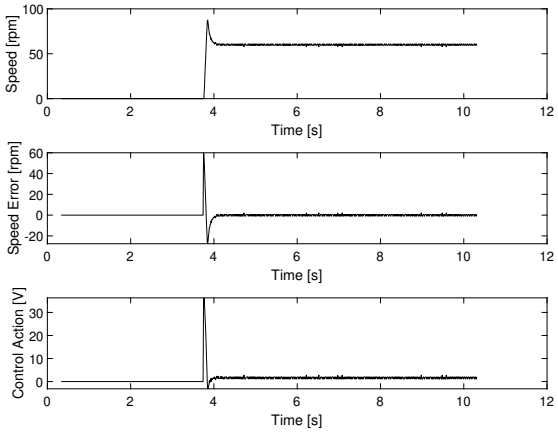
(a) Motor 1 (b) Motor 2

Figure 15: Forward and Brake 60 $[rpm]$

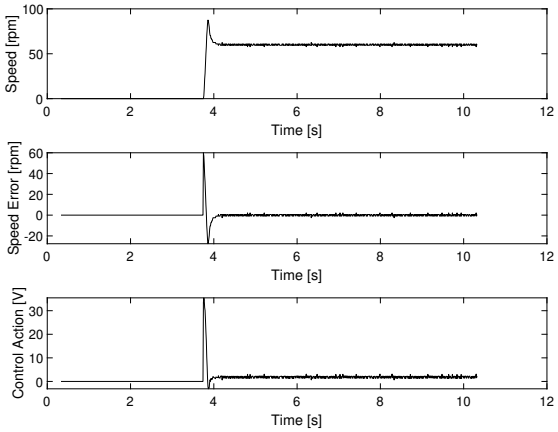


(a) Motor 1 (b) Motor 2

Figure 16: Forward and Brake -60 $[rpm]$

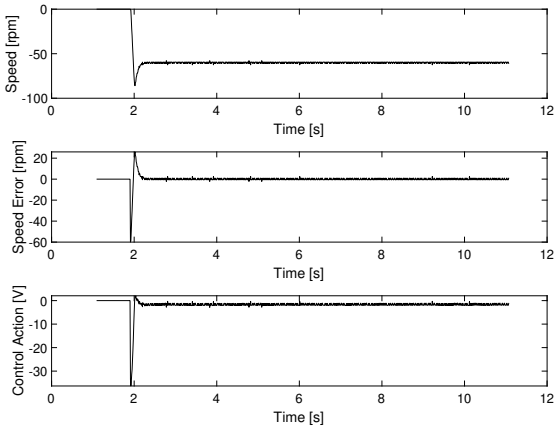


(a) Motor 1

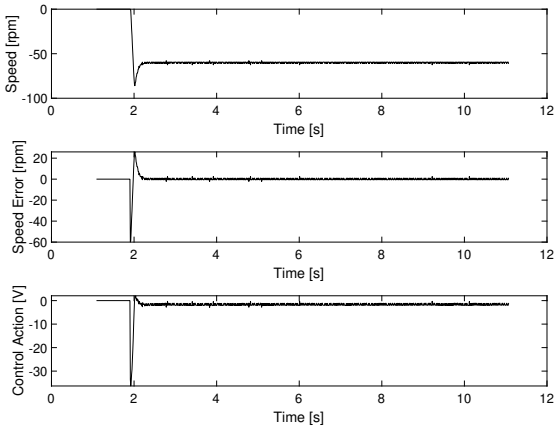


(b) Motor 2

Figure 17: Forward and Coast 60 $[rpm]$

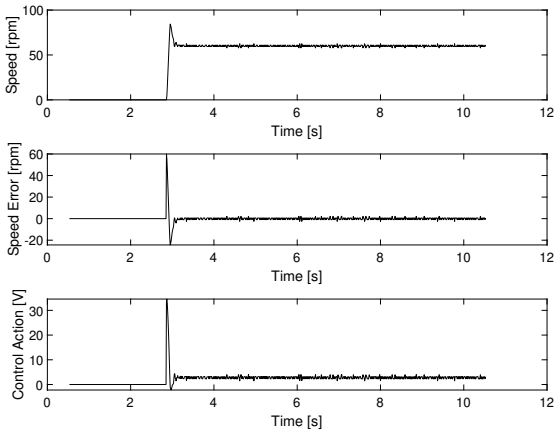


(a) Motor 1

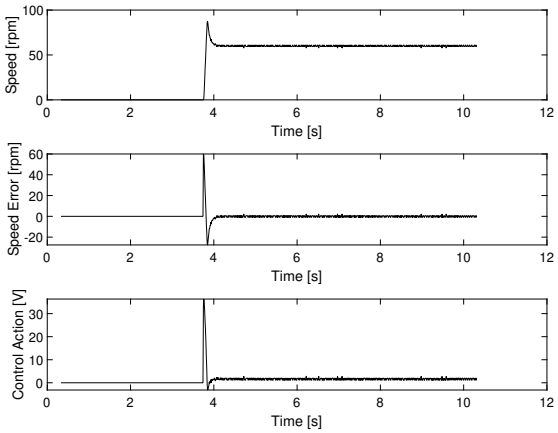


(b) Motor 2

Figure 18: Forward and Coast -60 $[rpm]$



(a) Forward and Brake



(b) Forward and Coast

Figure 19: Comparison Between Forward/Brake and Forward/Coast Modes

For the sake of simplicity, Figure 19 reports again the plots already shown in Figures 15a and 17a, corresponding to Motor 1 with a 60 RPM step reference, under Forward/Brake and Forward/Coast modes, respectively. These plots are placed side-by-side here to enable a more immediate and clear comparison of the two control strategies.

From the figure, it can be observed that the two approaches produce very similar dynamic responses. Both configurations exhibit comparable rise times, settling behavior, and steady-state performance. Nonetheless, small differences can still be noticed: the Forward/Brake mode (Figure 19a) results in a noisier speed signal

once the reference is reached, whereas the Forward/Coast mode (Figure 19b) results in a slightly smoother control action. The behaviour of the second method can be attributed to the fact that the motor is allowed to coast, resulting in a more gradual speed adjustment.

An important aspect visible in both cases is the control action, which exhibits a sharp peak reaching up to 35 V immediately after the step input. However, the actuator is limited to 12 V, and thus enters saturation. During this saturation phase, the integrator continues to accumulate error, resulting in integral windup, which contributes to the overshoot observed in the speed response. This issue could be mitigated by implementing an anti-windup scheme in the controller.

4 Laboratory 4, Line following

The goal of this laboratory activity was to implement a simple algorithm for line following using our TurtleBot. The robot was equipped with a line sensor array, which allowed it to detect the presence of a line on the ground. The line sensor array consisted of multiple infrared sensors that could detect the contrast between the line and the background surface. In our case the sensor was placed on the front side of the rover. The robot was programmed to follow the line by adjusting its speed and direction based on the sensor readings. In particular we implemented two algorithms:

- A simple initial algorithm that used the sensor readings to determine whether the robot was on the line, off to the left, or off to the right. Based on this information, the robot adjusted its speed and direction to stay on the line.
- A more advanced algorithm, i.e. the yaw controller, which used the sensor readings to calculate the error in the robot's position relative to the line. The robot then adjusted its speed and direction based on this error, allowing it to follow the line more accurately.

4.1 Theoretical background

In this section we will describe the model of the robot used in this laboratory activity. In particular, we will focus on the line sensor array and on the kinematic model of the robot.

4.1.1 Line sensor array

Our line sensor array consisted of the Pololu QTR reflectance sensors, whose modo operandis is described in Section 1.2.4. We remind that the microcontroller can read from the reflectance sensor by interfacing with a port expander, which is connected to the sensors via an I2C bus connection. An important parameter of the sensor line is the pitch, which is the distance between two adjacent sensors. The list below summarizes the main parameters of the sensor array used in this laboratory activity:

Parameter	**Value**
Sensor output	digital
Number of sensors (N)	8
Pitch (P)	8 mm or 0.008 m

Table 3

Furthermore, we worked under the following assumptions:

- The line is wide enough to activate at least two sensors at a time;
- There is always at least one sensor that is not active;
- A line is detected if one or more adjacent sensors are activate.

For our goal, it was important to define the line tracking error e_{SL}, which is the distance between the center of the robot and the center of the line, i.e.

$$f(x) = \begin{cases} e_{SL} = 0 & \text{when the line is at the center} \\ e_{SL} > 0 & \text{when the line is on the left side of the sensor} \\ e_{SL} < 0 & \text{when the line is on the right side of the sensor} \end{cases} \tag{6}$$

We define the sequence of active sensors as:

$$b_n = \begin{cases} 1 & \text{if sensor } n \text{ is active} \\ 0 & \text{if sensor } n \text{ is not active} \end{cases} \tag{7}$$

and the distance between the sensor at position n and the center of the sensor array as:

$$\lambda_n = \left(\frac{N-1}{2} - n \right) P, \quad n = 0, \ldots, N-1 \tag{8}$$

The line tracking error can then be computed as the weighted average of the distances of the activate sensors from the center of the sensor array, i.e.:

$$e_{SL} = \frac{\sum_{n=0}^{N-1} b_n \lambda_n}{\sum_{n=0}^{N-1} b_n} \tag{9}$$

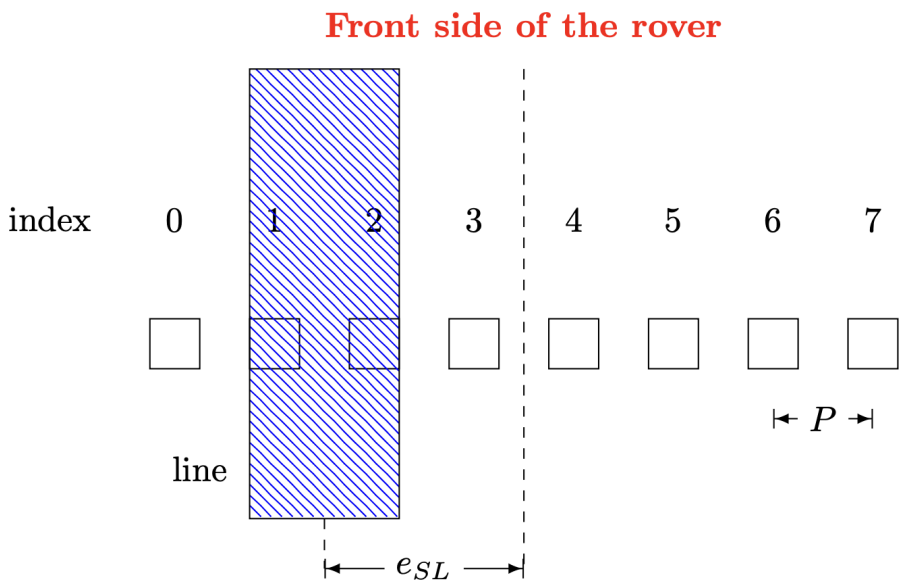


Figure 20: Line tracking error

4.1.2 Kinematic model

In Figure 21 is shown the kinematic model of the robot used in this laboratory activity, whose mechanical parameters are reported in Table 4.

Parameter	Value
Wheel distance (D)	0.165 m
Distance between the sensor and the vehicle's centre (H)	0.085 m
Wheel radius (r)	0.034 m

Table 4

The robot has two wheels that can be controlled independently, and in the following we assume a pure roll condition, i.e. the wheels do not slip on the ground. The relations that connect the linear velocity of the wheels V_l and V_r with the angular velocities of the left and right wheels ω_l and ω_r are:

$$V_r = r \cdot \omega_r, \quad V_l = r \cdot \omega_l \tag{10}$$

where r is the radius of the wheels. The rotation of the robot is around the Instantaneous Center of Rotation (ICR), which is the point where the robot is not moving. The ICR is located at a distance R from the center of the robot. While, the distance between the two wheels is denoted as D. It holds that:

$$V_r = \omega(R + D/2), \quad V_l = \omega(R - D/2) \tag{11}$$

and by subtracking these two equations we obtain:

$$\omega = \frac{V_r - V_l}{D} \tag{12}$$

The angle ψ between the global frame and the robot's frame is equal to the angle between the x-axis and the line connecting the ICR to the center of the robot, i.e. ω. Therefore, the yaw rate of the robot is equal to ω, i.e.:

$$\dot{\psi} = \frac{V_r - V_l}{D} \tag{13}$$

Then, the linear velocity of the robot can be computed as the average of the linear velocities of the two wheels, i.e.:

$$V = \frac{V_r + V_l}{2} \tag{14}$$

And, finally, we derive the formulas which take into account both the longitudinal speed of the wheels and the yaw rate:

$$\begin{cases} V_r = V + \dot{\psi}\frac{D}{2} \\ V_l = V - \dot{\psi}\frac{D}{2} \end{cases} \tag{15}$$

from which it immediately follows

$$\begin{cases} \omega_r = \frac{V + \dot{\psi}\frac{D}{2}}{r} \\ \omega_l = \frac{V - \dot{\psi}\frac{D}{2}}{r} \end{cases} \tag{16}$$

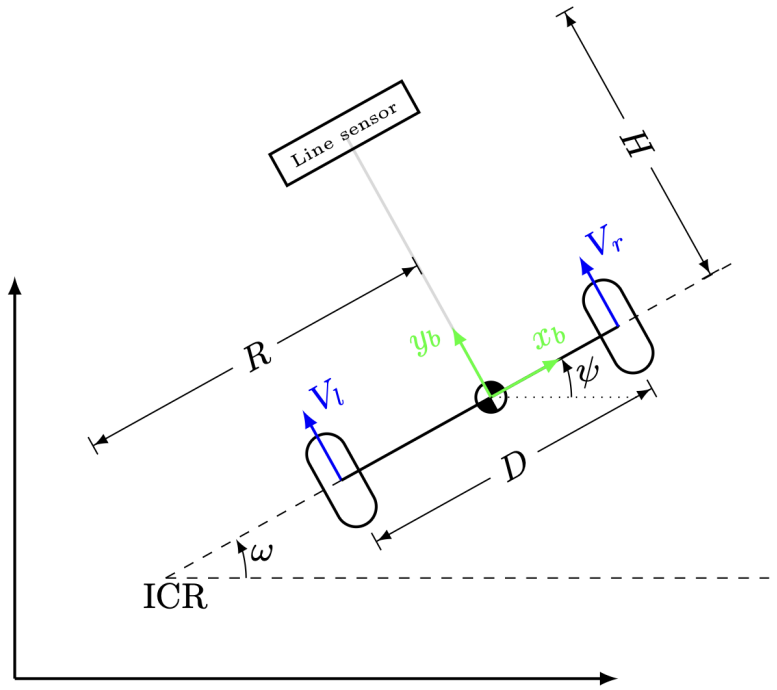


Figure 21: Kinematic model

4.1.3 Yaw error

At this point, we need to define the yaw error, which is the difference between the desired yaw angle (ψ_{ref}) and the actual yaw angle (ψ) of the robot. The yaw error can be expressed as:

$$\psi_{err} = \psi_{ref} - \psi \tag{17}$$

The relationship between the yaw error and the line tracking error can be expressed as follows:

$$\psi_{err} = \arctan\left(\frac{e_{SL}}{H}\right) \approx \frac{e_{SL}}{H} \tag{18}$$

Where the last equation exploits the small angle approximation, which is valid for small values of e_{SL}.

4.2 System configuration

This section describes the hardware components, architecture, and configuration of the mobile robot system used for yaw control and line following.

The hardware components are the same described in Section 1.2.

The control structure of the system, used for the implementation of the yaw controller, is divided into two layers:

- Low-level layer: responsible for maintaining the desired velocities of the left and right wheels using PI controllers.

- High-level layer: computes the yaw error from the sensor data and applies a proportional correction to the wheel speed reference, effectively steering the robot back onto the path.

When the robot deviates from the center of the line, the yaw error increases. The proportional yaw controller reacts by adjusting the difference between the left and right wheel speeds, thereby rotating the robot toward the line. This hierarchical control scheme ensures smooth and responsive tracking of the line.

For the design of the yaw controller, we considered as input the yaw error (ψ_{err}) and the output was the control signal for the robot's wheels, i.e. the yaw rate ($\dot{\psi}$), as shown in Figure 22.

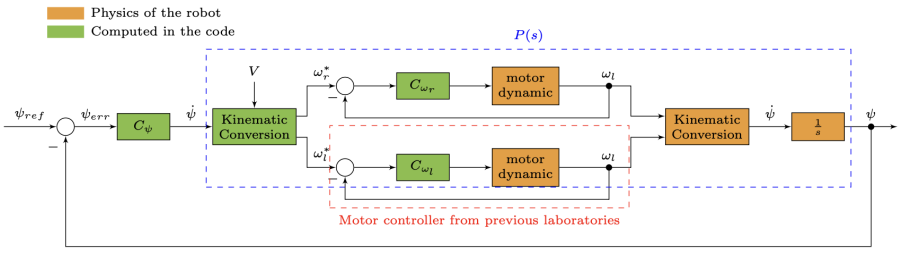


Figure 22: Yaw controller

4.3 Code implementation

4.3.1 Implementation of the simple controller

As initial approach we implemented a simple controller with the goal of keeping the robot on the line. The controller was designed to to adjust the direction of the robot, by increasing the rotation speed of one wheel and decreasing the other, based on the readings of the line sensor array.

The global variables used in the code are shown in Code 17. These include the speeds of the motors (in RPM), reference values, control errors, and proportional-integral control terms for each motor. Gains Kp and Ki are used in the PI controller to regulate the motor speed based on the error. Variables are defined separately for motor 1 (controlled via timer 3) and motor 2 (controlled via timer 4).

Listing 17: Global variables

```
// motor 1
float TIM3_speed; // Speed of motor 1 in RPM
float reference_speed = 60; //RPM
float speed_error;
static float u_int; // Integral term for speed control
float u_p; // Proportional term for speed control
float u_pi; // Proportional-Integral term for speed control
float Kp = 0.5;   // Proportional gain
float Ki = 6;     // Integral gain
uint32_t TIM3_CurrentCount;
int32_t TIM3_DiffCount;
static uint32_t TIM3_PreviousCount = 0;
float duty;

// motor 2
float TIM4_speed; // Speed of motor 2 in RPM
float reference_speed2 = 60; //RPM
float speed_error2;
static float u_int2; // Integral term for speed control
float u_p2; // Proportional term for speed control
float u_pi2; // Proportional-Integral term for speed control
float Kp2 = 0.5;   // Proportional gain
float Ki2 = 6;     // Integral gain
uint32_t TIM4_CurrentCount;
int32_t TIM4_DiffCount;
static uint32_t TIM4_PreviousCount = 0;
float duty2;
```

First of all, we checked the status of the line sensor, and we saved it in an array, where each element represents the status of a sensor (active [1] or not active [0]).

Listing 18: Status sensor line

```
uint8_t sensor_line;
HAL_StatusTypeDef status_sens_line = HAL_I2C_Mem_Read(&hi2c1,
    SX1509_I2C_ADDR1 << 1, REG_DATA_B, 1, &sensor_line, 1, I2C_TIMEOUT);
for (int i = 0; i < 8; i++) {
    if ((sensor_line >> i) & 0x01) {
        active_pins[i] = 1;
    } else {
        active_pins[i] = 0;
    }
}
```

Next, inside the `HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)` function, we read the array `active_pins` and we implemented the simple controller. We decided to monitor only the pins on the far left (`active_pins[0..2]`) and the far right (`active_pins[5..7]`) to determine if the robot is drifting off the line. Based on which pins are active, we adjust the reference speed of one motor to induce a turning behavior (see Code 19). It is important to note that the `if` statements are not mutually exclusive, so if multiple conditions are satisfied, the last one overrides the previous ones. This allows for a graded control based on how far the robot is from the center. The logic focuses on turning by modifying only one of the two motors. For instance, if the robot needs to turn left, only the speed of motor 2 is increased, while motor 1 remains unchanged (default speed of 0 unless otherwise specified).

Listing 19: Simple algorithm

```
    //turn left
    if(active_pins[0])
        reference_speed2 = 10;
    else if(active_pins[0] && active_pins[1])
        reference_speed2 = 25;
    else if(active_pins[0] && active_pins[1] && active_pins[2])
        reference_speed2 = 37;

    //turn right
    if(active_pins[7])
        reference_speed = 10;
    else if(active_pins[7] && active_pins[6])
        reference_speed = 25;
    else if(active_pins[7] && active_pins[6] && active_pins[5])
        reference_speed = 37;
```

Note: The computation of motor speed from the encoder, the evaluation of the control error, and the implementation of the PI controller are already described in detail in Section 3.3.

4.3.2 Implementation of the yaw controller

To the global variables defined in Listing 17, we added the following variables for the yaw controller:

Listing 20: Global variables for yaw control

```
float e_num,e_den,e_sl; // line tracking error
float psi_err; // yaw error
float V, w_1, w_2; // speed reference after the controller yaw
float kp_yaw = 7; // proportional gain for yaw control
float yaw_rate;
float lambda[8]; // distance between the sensors and the center of the sensor array
```

As for the previous approach, inside the `HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)` function, we read the status of the line sensors and saved it in an array, see Listing 18. And we used it to compute the line tracking error e_{SL}, as shown in Listing 21, where we resorted to Equation (8), (9) and (18).

Listing 21: Line tracking error

```
e_num = e_den = 0;

for(int i=0;i<N;i++){
    lambda[i] = ((N-1)/2 - i)*P; // distance between sensor n and the center
    e_num += active_pins[i]*lambda[i]; // distance between sensor n and the center
    e_den += active_pins[i];
}
if(e_den == 0)
    e_sl = 0;
else
    e_sl = e_num / e_den;

psi_err = e_sl / H;
```

The yaw rate is calculated as a proportional term of the psi error as shown in Listing 22, which is then used to adjust the wheel speeds accordingly. Indeed, we computed the desired wheel speeds w_1 and w_2 based on the reference speed and the yaw rate, using Equation (16), where the reference speed was set to 30 RPM. Next, we used them to compute the speed error for each motor: `speed_error = w_1 - TIM3_speed;` for motor 1 and `speed_error2 = w_2 - TIM4_speed;` for motor 2.

Listing 22: Yaw control law

```
yaw_rate = kp_yaw * psi_err;
V = (reference_speed*RPM2RADS)*r; //RADS
w_1 = ((V + yaw_rate*(D/2))/r)*RADS2RPM; // Right wheel speed
w_2 = ((V - yaw_rate*(D/2))/r)*RADS2RPM; // Left wheel speed
```

Finally, as detailed in Section 2.3.3, the Wi-Fi datalogger is used to acquire and store experimental data, enabling accurate offline processing and analysis for performance evaluation.

4.4 Results

In this section we will present the results of the laboratory activity, focusing on the performance of the implemented line following algorithms. We will analyze only the results obtained with the yaw controller, as it was the most advanced approach and yielded the best performance.

Figures 23 and 24 show the performance of the yaw controller for Wheel 1 and Wheel 2, respectively, over a 6-second time window. For each wheel, five plots are reported:

- the actual and reference wheel speeds,
- the speed tracking error,
- the control input (voltage) delivered to the motor,
- the resulting yaw rate (in rad/s),
- the yaw angle error (psi error, in rad).

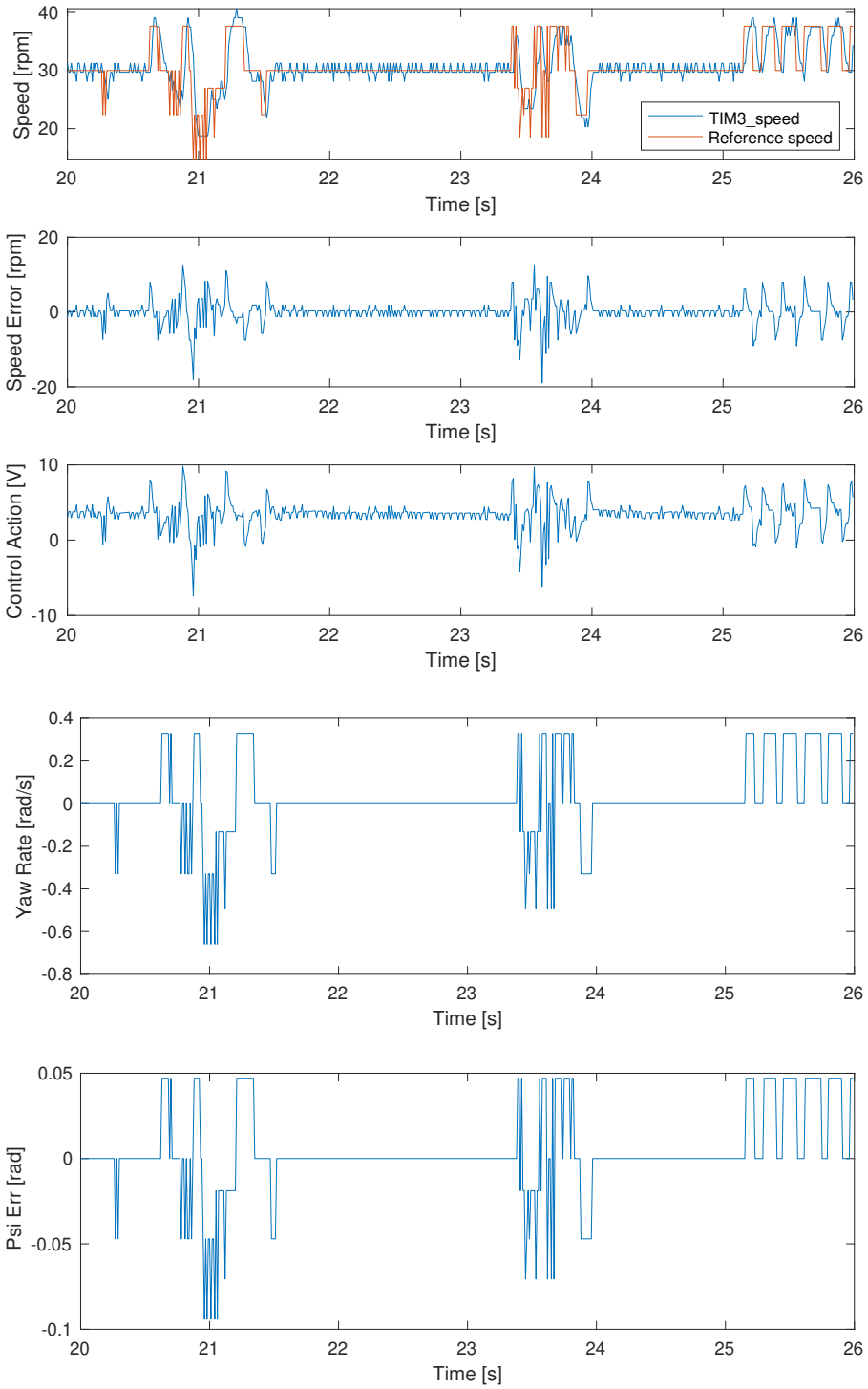


Figure 23: Yaw controller results for wheel 1

From the first subplot, it is evident that the controller for Wheel 1 is able to track the reference speed with reasonable accuracy, especially during steady-state segments. Transient deviations occur when the reference signal changes abruptly, leading to observable overshoots and undershoots. These are reflected in the speed error plot, where oscillations are more pronounced in the initial and final parts of the time window.

The third subplot shows the control action in volts. The control signal remains mostly within the saturation limits of ± 10V, indicating that the controller operates within feasible bounds. The presence of sharp control changes suggests that the system is responding to rapid reference changes or disturbances, consistent with what is expected in a dynamic line-following scenario.

The yaw rate plot demonstrates that the robot frequently adjusts its orientation to maintain the desired trajectory, with several peaks and zero-crossings indicating turns or corrections. Importantly, the psi error remains mostly bounded within ± 0.1 rad, indicating effective heading control throughout the experiment.

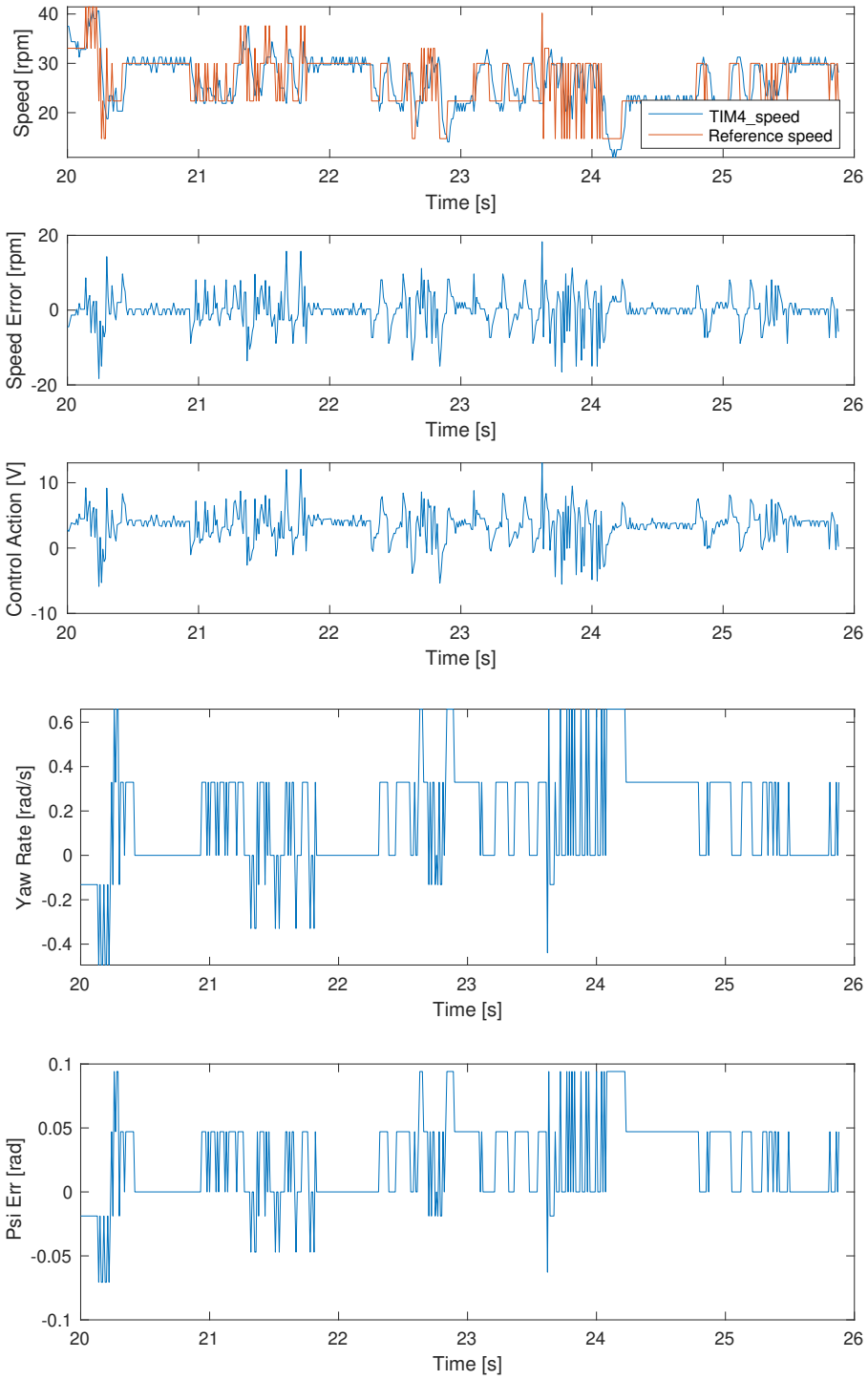


Figure 24: Yaw controller results for wheel 2

The behavior of Wheel 2 follows a similar trend, although some differences can be observed. The reference tracking in the first subplot appears slightly more oscillatory, likely due to asymmetries in the mechanical response or load distribution. The speed error is larger during some transitions, which translates to more aggressive control actions in the third subplot.

Despite this, the control voltage remains within saturation bounds and demonstrates good responsiveness. The yaw rate and psi error plots again confirm that the controller achieves continuous correction of orientation. Notably, psi error remains within the same tight bounds observed for Wheel 1, reinforcing the robustness of the overall yaw control strategy.

Overall, the yaw controller successfully maintains trajectory alignment by dynamically adjusting the individual wheel speeds. Both wheels contribute to heading correction, and the system demonstrates good performance in terms of responsiveness, error containment, and control effort. These results validate the effectiveness of the implemented controller in enabling stable and accurate line following under varying speed references and disturbances.

4.5 Conclusion

In this laboratory activity, we successfully implemented a line-following algorithm for the TurtleBot using a combination of low-level and high-level control strategies. The low-level controllers ensured precise wheel speed control, but the trajectory was not smooth, since the robot tended to oscillate around the line. To address this, we introduced a high-level yaw controller that adjusted the robot's heading based on the line tracking error. This approach significantly improved the robot's ability to follow the line smoothly and accurately. The results demonstrated that the yaw controller effectively minimized the yaw error, allowing the robot to maintain a stable trajectory.

We believe that further improvements can be made by tuning the controller parameters and increasing the velocities, which would allow the robot to follow the line more quickly while maintaining stability.